MFJA 3rd Floor Gazebo Harmonic Current Multi-Robot and Room 315 Kinematic Shuttle Report

Technical Project Report

April 20, 2026

Contents

Chapter 1

Purpose and Current Scope

This report documents the current accepted state of the `mfja_3rd_floor_gz` meta-repository. The project now combines two main layers:

- a reusable Gazebo Harmonic and ROS 2 Jazzy multi-robot simulation for the MFJA third floor;
- a kinematic-first Room 315 shuttle simulation for the flexible rail network.

The current shuttle work is intentionally kinematic. The shuttle does not use wheel contact, rail contact dynamics, or full rigid-body contact as the primary motion source. Instead, the rail path is represented as CSV geometry plus an explicit directed graph. The node computes the shuttle pose along rail segments and sends pose updates to Gazebo through `/world/<world_name>/set_pose`.

Dynamic shuttle work is not part of the current main version. The current main version focuses on:

- independent robot control through namespaced ROS topics;
- a clean ROS 2 meta-repository package structure;
- Room 315 kinematic shuttle routing;
- independent switch states;
- independent binary stoppers before switches;
- sensor-style approach events for the teaching workflow;
- per-shuttle ON/OFF commands;
- runtime shuttle spawning;
- explicit rejection when a requested start slot is occupied;
- simple center-distance collision avoidance between shuttles.

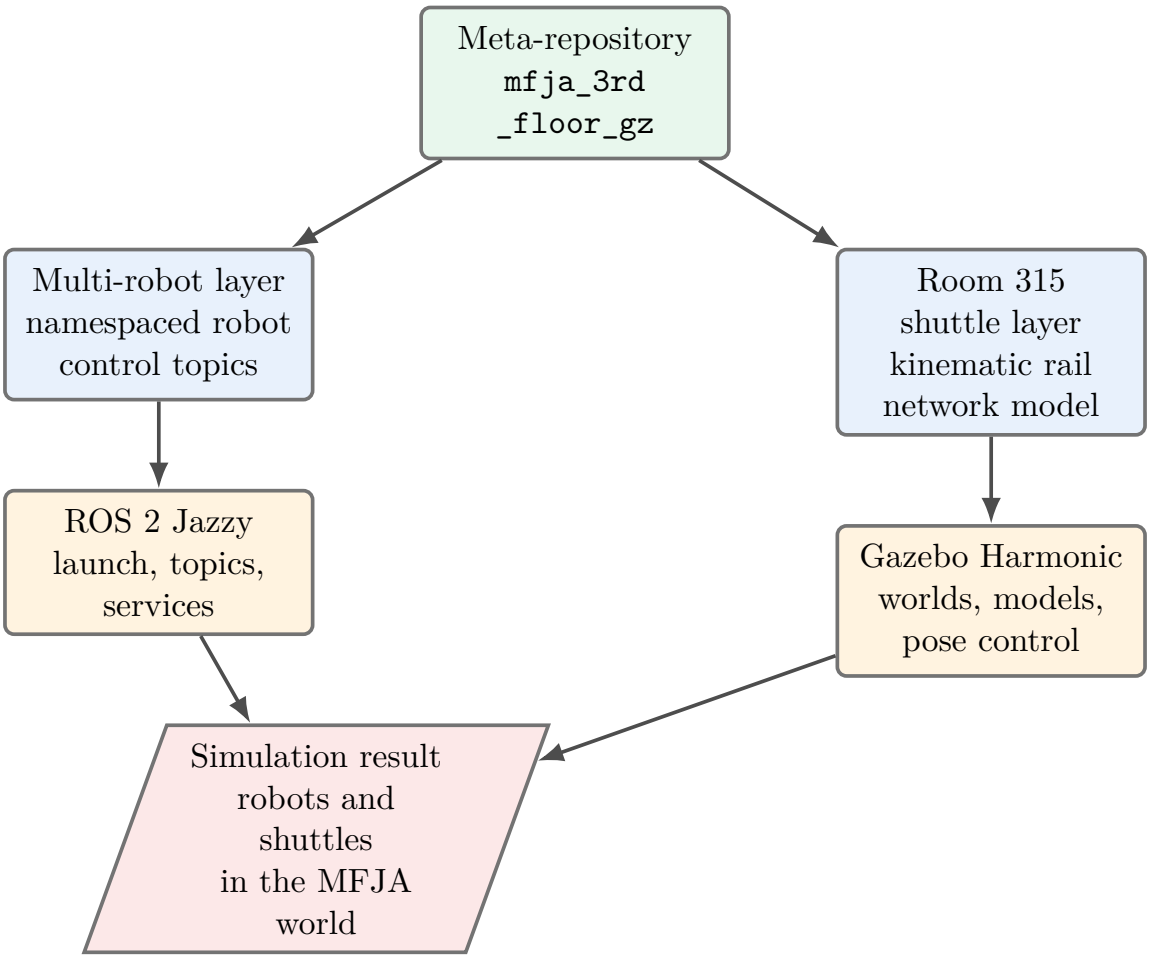

Figure 1.1: High-level project scope. The repository now contains a general multi-robot simulation layer and a Room 315 kinematic shuttle layer. Both are integrated through ROS 2 and Gazebo.

Chapter 2

Repository Architecture

2.1 Meta-repository layout

The Git repository root is intentionally not a ROS 2 package. The root contains documentation and repository-level files only. ROS packages live in subfolders:

- `mfja_3rd_floor_description`: shared Gazebo models, meshes, worlds, and URDF/SDF assets;
- `mfja_robot_control_config`: shared launch logic, bridge configuration, robot registries, Room 315 kinematic configuration, and control scripts;
- `mfja_room_315_bringup`: launch entry point for Room 315 only;
- `mfja_3rd_floor_bringup`: launch entry point for the full floor;
- `mfja_3rd_floor_gz`: umbrella compatibility package that keeps historical launch entry points available.

The root should not contain package-level files such as root `package.xml`, root `CMakeLists.txt`, root `models/`, root `worlds/`, root `launch/`, root `config/`, or root `CSV/`. Package-owned files belong inside the package that owns them.

2.2 Build command

The explicit `-base-paths` command is the recommended build method, because the repository contains several ROS 2 packages inside one Git repository.

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash

colcon build --base-paths \
  src/mfja_3rd_floor_gz/mfja_robot_control_config \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_description \
  src/mfja_3rd_floor_gz/mfja_room_315_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_gz \
  --packages-select \
  mfja_robot_control_config \
  mfja_3rd_floor_description \
  mfja_room_315_bringup \
  mfja_3rd_floor_bringup \
  mfja_3rd_floor_gz \
  --symlink-install

source install/setup.bash
```

Listing 2.1: Current meta-repository build command.

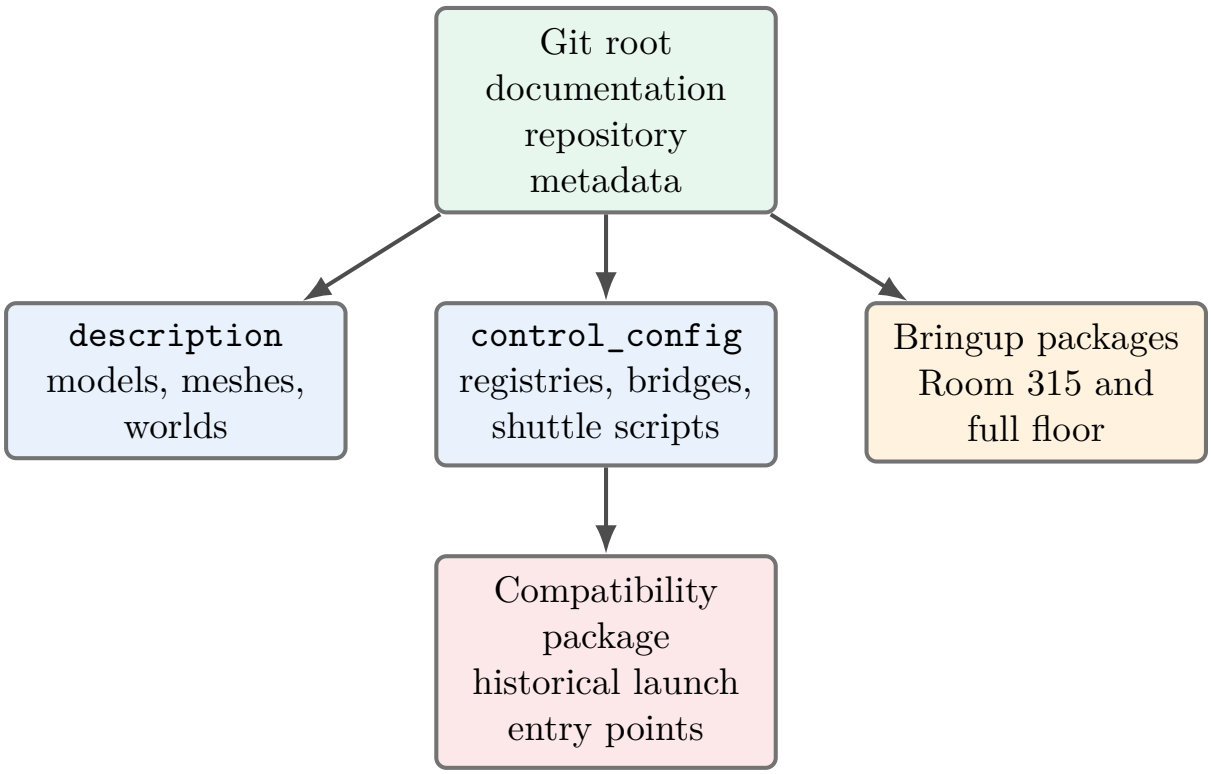

Figure 2.1: Repository ownership model. The root is not a ROS 2 package. Runtime assets live inside the package that owns them, which keeps the workspace buildable and easier to maintain.

Chapter 3

Multi-Robot Simulation Layer

3.1 Supported robots

The current simulation supports these robot instances:

Instance	Robot family	Primary command topic
`kuka1`	KUKA KR6 R900 sixx	`/kuka1/joint_trajectory`
`staubli1`	Staeubli TX2-60L	`/staubli1/joint_trajectory`
`yaskawa_hc10_1`	Yaskawa HC10	`/yaskawa_hc10_1/joint_trajectory`
`yaskawa_hc10dt_1`	Yaskawa HC10DT	`/yaskawa_hc10dt_1/joint_trajectory`
`tiago1`	TIAGo mobile manipulator	`/tiago1/joint_trajectory`, `/tiago1/cmd_vel`

Each robot uses its own namespace. A command sent to one namespace does not affect another robot.

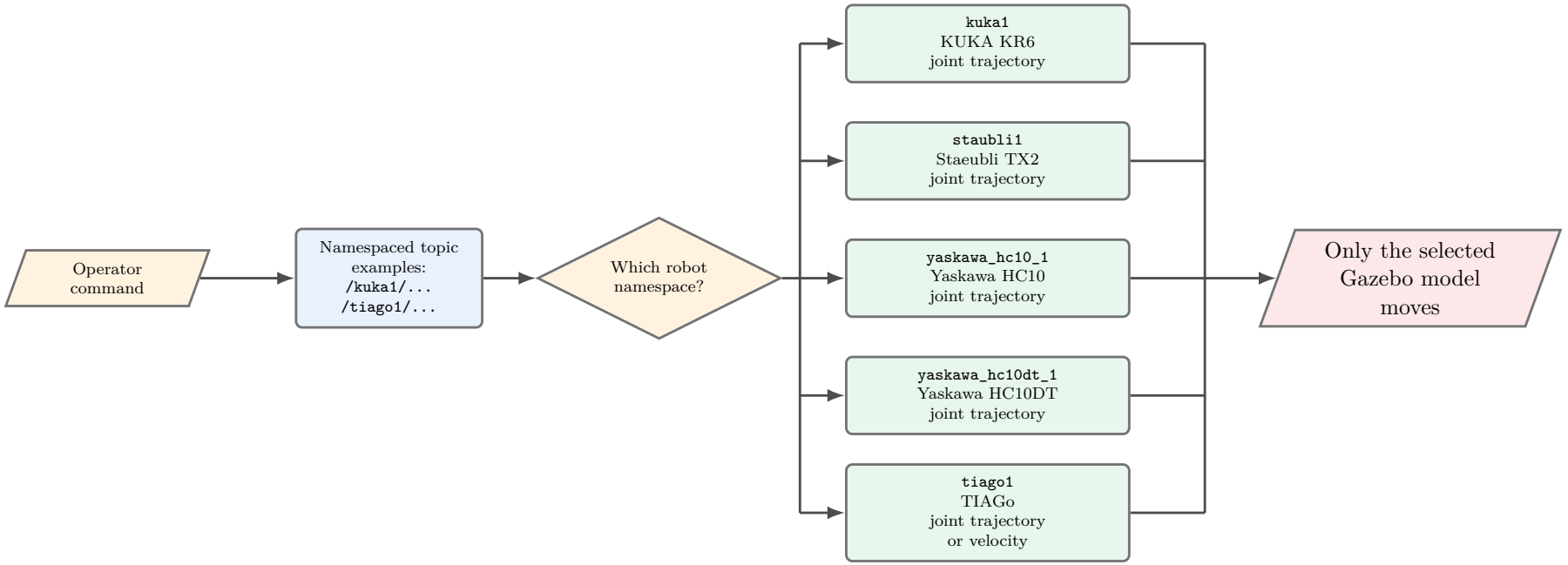

Figure 3.1: Namespaced robot control for all supported robot instances. Each command is routed through one robot namespace, so commanding one robot does not move the others.

3.2 Robot registries

Robot identity, model selection, pose, and activation are stored in YAML registries:

- `mfja_robot_control_config/config/robots.yaml` for the full floor;
- `mfja_robot_control_config/config/robots_room_315_only.yaml` for Room 315 only.

This makes robot deployment data-driven. A developer can adjust active robots and placements without editing the launch implementation.

3.3 Full floor launch

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=none \
  start_paused:=false \
  gui:=true
```

Listing 3.1: Launching the full floor.

3.4 Room 315 only launch

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=none \
  start_paused:=false \
  gui:=true
```

Listing 3.2: Launching Room 315 only.

3.5 Robot topic checks

```
ros2 topic list | grep -E '^/(kuka1|staubli1|yaskawa_hc10_1|yaskawa_hc10dt_1|tiago1)/'

ros2 topic info /kuka1/joint_trajectory
ros2 topic info /staubli1/joint_trajectory
ros2 topic info /yaskawa_hc10_1/joint_trajectory
ros2 topic info /yaskawa_hc10dt_1/joint_trajectory
ros2 topic info /tiago1/joint_trajectory
ros2 topic info /tiago1/cmd_vel
```

Listing 3.3: Checking robot control topics.

3.6 Example one-robot commands

```
ros2 topic pub --once /kuka1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_a1','joint_a2','joint_a3','joint_a4','joint_a5','joint_a6'], \
points: [{positions: [0.6,-1.0,1.1,0.0,0.6,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 3.4: KUKA example command.

```
ros2 topic pub --once /staubli1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1','joint_2','joint_3','joint_4','joint_5','joint_6'], \
points: [{positions: [0.1,0.4,-0.6,0.0,0.5,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 3.5: Staeubli example command.

```
ros2 topic pub -r 20 /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.15, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.20}}"
```

Listing 3.6: TIAGo base example command.

3.7 Synchronized multi-robot tool

The script `multi_robot_sync_demo.py` can publish coordinated commands from one ROS 2 process. It is better than starting several shell commands manually when multiple robots should begin a scenario together.

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py --list-joints
```

Listing 3.7: Inspecting supported synchronized joints.

```
ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
  --goal staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
  --trajectory-duration 4.0
```

Listing 3.8: Coordinated motion for selected robots.

Chapter 4

Room 315 Kinematic Shuttle Layer

4.1 Rail network model

The Room 315 rail system is modeled as a directed graph. CSV files contain the geometry of each one-way segment. The topology and routing rules are stored in:

`mfja_robot_control_config/config/room_315_kinematics/rail_network.yaml`

The source rail segments are stored in:

`mfja_robot_control_config/config/room_315_kinematics/raw_segments/`

Every segment is traversed forward only, from CSV `index=0` to the last row. Routing is not inferred from exact endpoint equality. Routing is decided by explicit tables in `rail_network.yaml`. If the shuttle reaches a segment end and no valid successor exists, it enters `FALLING` mode instead of teleporting or silently correcting the route.

4.2 Current start slots

Only four start slots are currently accepted:

Slot	Gazebo pose	Notes
1	`-14.95 -3.86 0.84 0 0 3.14`	upper rail
2	`-15.43 -3.86 0.84 0 0 3.14`	upper rail
3	`-15.24 -5.54 0.84 0 0 0`	lower rail
4	`-14.77 -5.54 0.84 0 0 0`	lower rail

Runtime add commands are rejected if the selected start slot is occupied. This is different from collision avoidance: the shuttle is not created at all when the requested start slot is blocked.

4.3 Starting the shuttle node

The same node works in Room 315 only and in the full floor. The key parameter is `gazebo_world_name`.

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=room_315_only \
  -p shuttle_count:=4 \
  -p start_slots:=1,2,3,4 \
  -p enable_gazebo_set_pose:=true \
  -p enable_gazebo_spawn:=true \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

Listing 4.1: Starting four shuttles in Room 315 only.

```
ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=mfja_3rd_floor \
  -p shuttle_count:=4 \
  -p start_slots:=1,2,3,4 \
  -p enable_gazebo_set_pose:=true \
  -p enable_gazebo_spawn:=true \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

Listing 4.2: Starting the same shuttle node in the full floor.

4.4 Switch primitive

Switch routing is controlled by:

`/room_315/switch_states`

The switch primitive has two logical states:

- `G`: big-loop branch;
- `S`: small-loop branch.

Aliases such as `BIG`, `LARGE`, and `SMALL` are also accepted by the command layer.

```
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=G A2=S A3=G A4=S'}"
```

Listing 4.3: Switch commands.

4.5 Stopper primitive

Stopper logic is independent from switch logic. A stopper is a binary primitive:

- `0`, `OPEN`, `RELEASE`, `OFF`: the stopper is open and shuttles may pass;
- `1`, `STOP`, `CLOSED`, `ON`: the stopper stops a shuttle before the switch.

The stopper command topic is:

`/room_315/stopper_states`

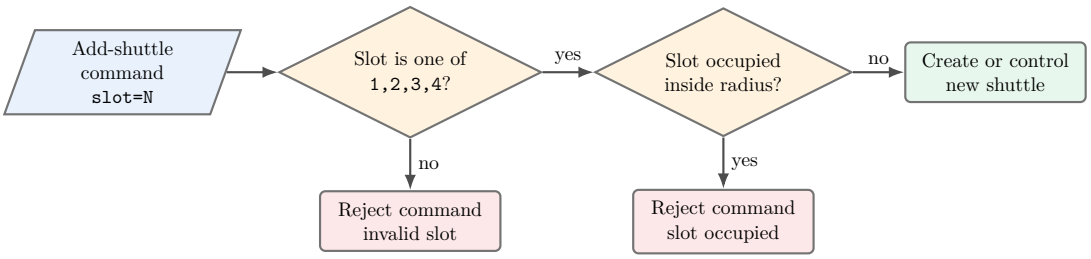

Figure 4.1: Start-slot acceptance logic. A shuttle is added only when the requested slot exists and the start position is clear.

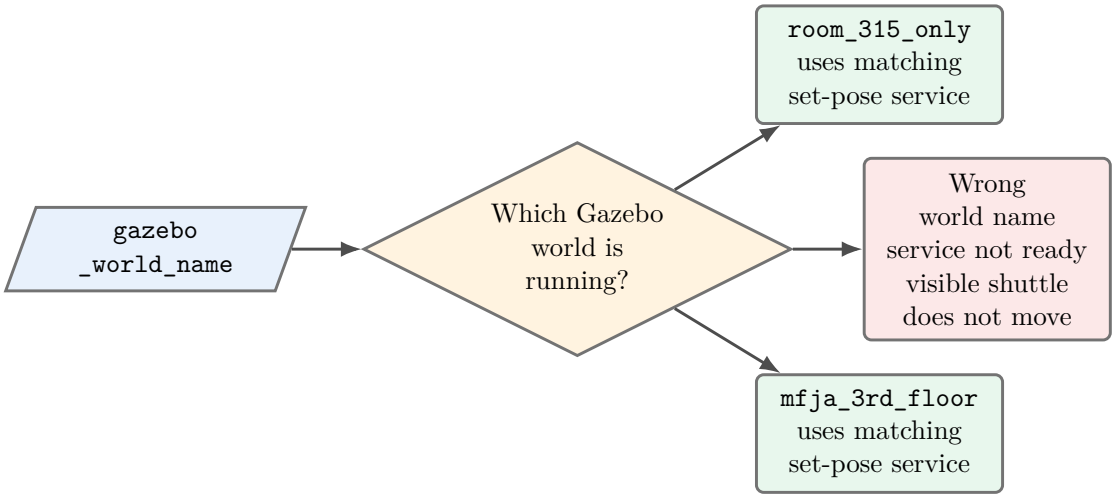

Figure 4.2: Gazebo world-name selection. The shuttle node can publish ROS topics without Gazebo moving anything if the set-pose service name does not match the running world.

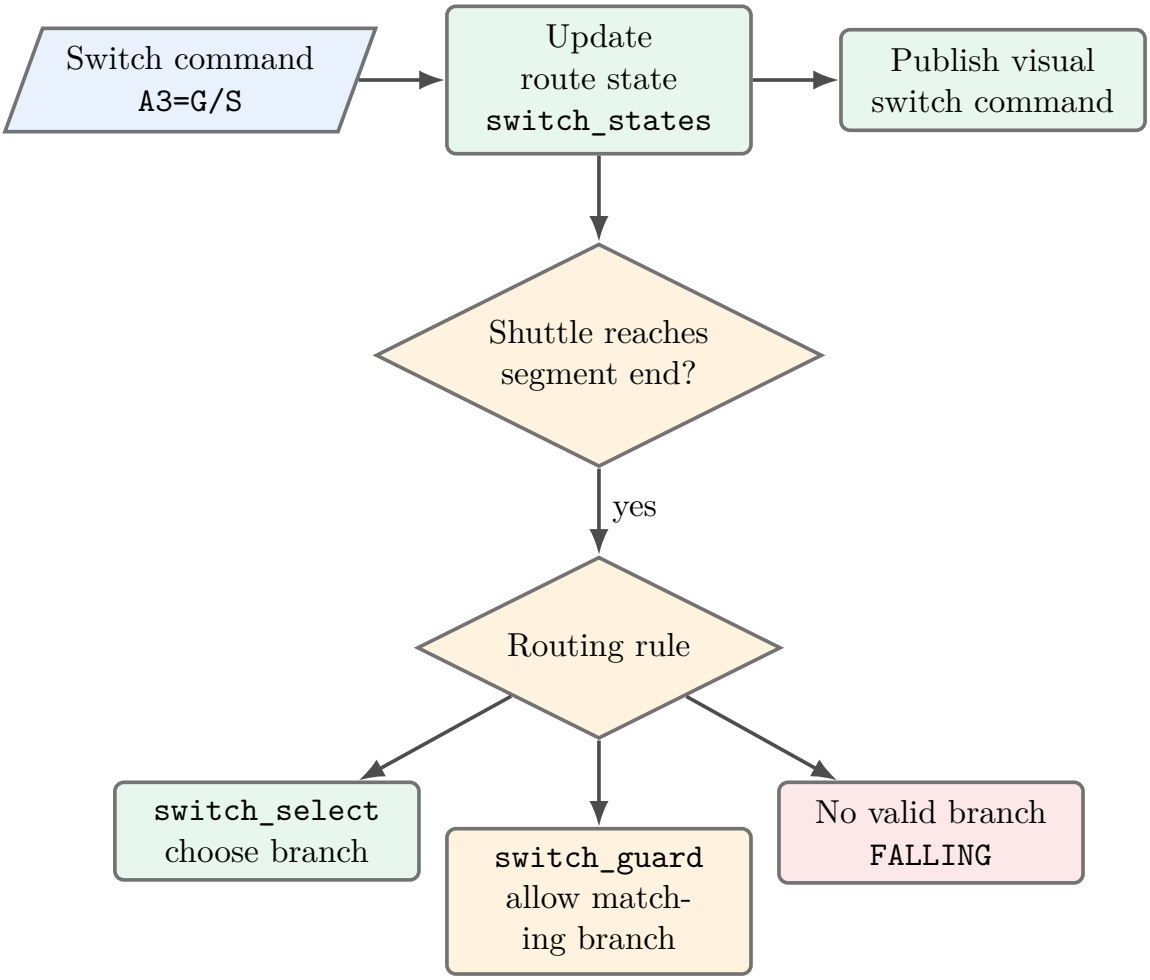

Figure 4.3: Switch command flow. The same command updates route logic and the visual switch. The selected route is used only when the shuttle reaches a segment boundary.

The current logical stoppers are:

Stopper	Before switch	Stop segments
A1	A1	A14
A2	A2	A12E, A12I
A3	A3	A23
A4	A4	A34E, A34I

```
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=1'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=0'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'ALL=1'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'ALL=0'}"
```

Listing 4.4: Stopper commands.

4.6 Sensor workflow

The educational workflow required by the project is:

`sensor -> stop shuttle -> move switch -> unstop shuttle`

The sensor-style approach events are published on:

`/room_315/sensors/switch_approach`

```
ros2 topic echo /room_315/sensors/switch_approach
```

Listing 4.5: Watching approach sensor events.

Each message is JSON text. The field `sensors` is a list because several shuttles can trigger approach sensors at the same time. The field `entity_name` identifies which shuttle the event belongs to.

```
{
  "sensors": [
    {
      "before_switch": "A3",
      "distance_m": 0.247,
      "entity_name": "room315_shuttle_4",
      "segment": "A23",
      "sensor": "A3_APPROACH",
      "stopper": "A3",
      "stopper_state": "0"
    }
  ]
}
```


Listing 4.6: Example approach sensor event.

This means that `room315_shuttle_4` is on segment `A23`, approaching switch `A3`, and is about `0.247 m` before the A3 stop point. If the distance decreases over time, the shuttle is moving toward that stopper.

4.7 Manual stop-switch-release workflow

```
ros2 topic echo /room_315/sensors/switch_approach
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=1'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=S'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=0'}"
```

Listing 4.7: Manual stopper and switch workflow.

The stopper stops the shuttle before the switch. The switch can then be moved while the shuttle is waiting. Opening the stopper releases the shuttle so it continues according to the new switch state.

4.8 Per-shuttle ON/OFF control

Each shuttle can be independently enabled or disabled through:

`/room_315/shuttle/control_cmd`

Disabling a shuttle keeps the model in place and stops its kinematic motion. Enabling it again lets it continue from the current rail segment and arc-length position.

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=OFF
    '}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=ON'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=ON'}"
```

Listing 4.8: Per-shuttle enable and disable commands.

4.9 Runtime add behavior

New shuttles can be requested through:

`/room_315/shuttle/add_cmd`

```
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=3'}"
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'entity=room315_shuttle_5
     slot=3 speed=0.2'}"
```

Listing 4.9: Runtime shuttle add commands.

If the requested start slot is occupied, the node rejects the command and does not create a new shuttle. This behavior was chosen because the expected operational rule is explicit refusal, not silent insertion followed by collision avoidance.

4.10 Collision avoidance

Collision avoidance remains separate from start-slot rejection and from stoppers. It is enabled by default:

- `enable_collision_avoidance=true`
- `shuttle_collision_distance_m=0.33`

If a moving shuttle gets too close to another active shuttle, it enters `WAITING` at the last safe pose instead of merging through the other shuttle.

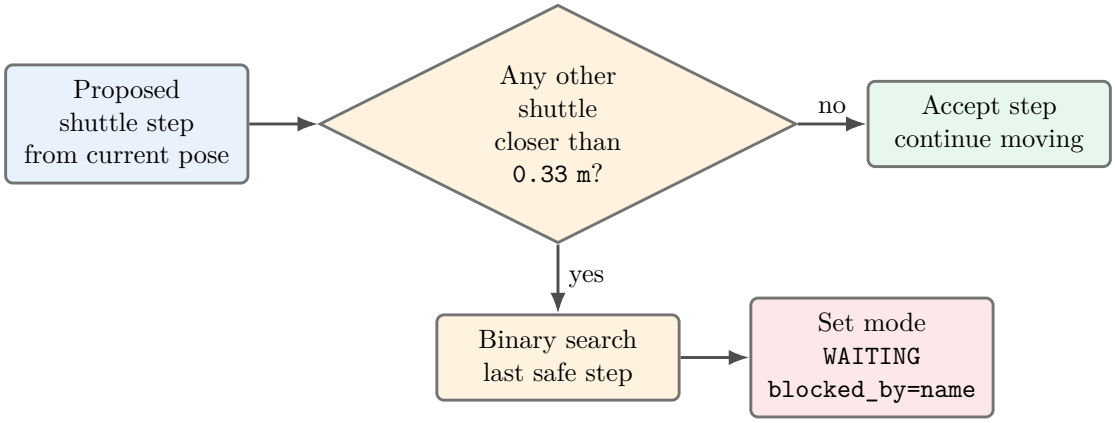

Figure 4.4: Collision avoidance logic. The node does not use contact dynamics in this phase; it prevents overlap by stopping at the last safe kinematic pose.

Chapter 5

Validation Procedures

5.1 CSV and graph validation

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_csv_preprocessor.py
ros2 run mfja_robot_control_config room_315_network_validator.py
ros2 run mfja_robot_control_config room_315_segment_plot.py
```

Listing 5.1: Room 315 preprocessing and validation commands.

The expected validation result is:

```
Status: PASS (0 warnings)
```

5.2 Runtime topics to inspect

```
ros2 topic echo /room_315/shuttle/state --once
ros2 topic echo /room_315/sensors/switch_approach --once
ros2 topic echo /room_315/shuttle/pose_cmd --once
ros2 topic echo /room_315/shuttles/room315_shuttle_3/pose_cmd --once
ros2 topic echo /mfja/conveyor/switch_states --once
ros2 topic list | grep room_315
```

Listing 5.2: Runtime state and debug topics.

In the shuttle state output, the most important fields are:

- `entity_name`: which shuttle the state describes;
- `mode`: `MOVING`, `WAITING`, or `FALLING`;
- `enabled`: whether the shuttle is allowed to move;
- `stopped_by`: stopper name or `DISABLED`;
- `blocked_by`: another shuttle that caused collision avoidance;
- `switch_states`: current route switch states;
- `stopper_states`: current stopper states.

5.3 Expected negative tests

```
# This should be rejected if slot 1 is still occupied.
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=1'}"

# This should always be rejected because only slots 1, 2, 3, and 4 are valid.
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=5'}"
```

Listing 5.3: Negative tests for start-slot rules.

The first command should produce an occupied-slot error when a shuttle is still inside the start-slot occupancy radius. The second command should report that the valid slots are `1`, `2`, `3`, and `4`.

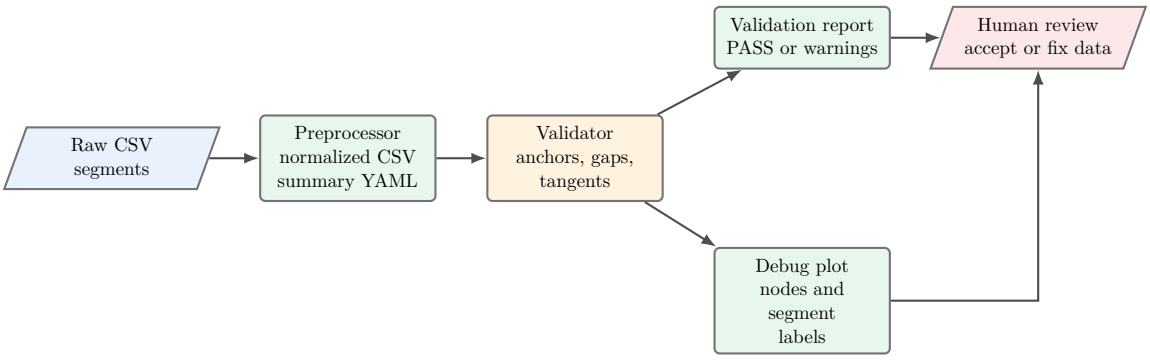

Figure 5.1: Validation workflow. The plots are not decorative; they are the visual check that the CSV geometry, anchors, and directed graph agree with the physical rail layout.

Chapter 6

Current Operational Notes

- Use `gazebo_world_name:=room_315_only` for the isolated Room 315 world.
- Use `gazebo_world_name:=mfja_3rd_floor` for the full-floor world.
- If `Gazebo set_pose service is not ready yet` appears, check the world name and `ros2 service list`.
- If a shuttle stops with `stopped_by` set to a stopper name, open that stopper through `/room_315/stopper_states`.
- If a shuttle stops with `stopped_by=DISABLED`, enable it again through `/room_315/shuttle/control_cmd`.
- If a shuttle enters `FALLING`, the graph has no valid successor for the current switch configuration.
- If a switch moves visually but routing does not change, send commands to `/room_315/switch_states`, not directly to the visual switch topic.

Chapter 7

Detailed Implementation Walkthrough

This chapter explains how the current implementation is organized internally. It is written as a code-oriented engineering walkthrough, not only as an operator guide. The goal is to make the current system understandable enough that another developer can maintain it, extend it, or debug it during a live demonstration.

7.1 Design rule: topology is explicit, geometry is only geometry

The most important design decision in Room 315 is that CSV geometry is not used as the routing authority. CSV files describe points along each rail segment. They are used to compute the shuttle pose and yaw at an arc-length position. They are not used to guess which segment comes next by comparing endpoints.

The authoritative route logic lives in `mfja_robot_control_config/config/room_315_kinematics/rail_network.yaml`. That file declares the nodes, segment start and end nodes, switch rules, fixed transitions, stopper locations, and start slots. This prevents fragile behavior caused by tiny endpoint mismatches in the CSV files.

7.2 How the rail paths were built from CSV files

The Room 315 shuttle path was not drawn as one large loop in code. It was built as a measured, directed rail network from CSV segment files. The CSV files are the geometric input. The YAML network is the topological input. The Python core combines both at runtime.

The complete path-building workflow is:

```
measured rail points
  -> CSV files with columns index,x,y,z
  -> one file per directed rail segment
  -> duplicate-point cleanup and normalization
  -> endpoint snapping to named anchors
  -> rail_network.yaml topology and routing
  -> validation report and debug plots
  -> arc-length geometry in the Python core
  -> ROS 2 node publishes Gazebo poses
```

Listing 7.1: CSV-to-runtime workflow used for Room 315.

Step 1: split the physical rail into directed segments

The physical rail was split at the switch breakpoints. Each piece between two important connection points became one segment. The current accepted segment set is:

```
A1G A1S A2G A2S A3G A3S A4G A4S A12E A12I A14 A23 A34E A34I
```

Every segment has its own CSV file. Each CSV uses this schema:

```
index,x,y,z
0,-12.411...,...
1,-12.412...,...
...
```

The `index` column preserves the intended travel order. The shuttle travels only from `index=0` toward the last row of that file. That rule is important because the same physical rail shape could otherwise be interpreted in two directions. The current network does not allow reverse traversal.

Step 2: keep CSV geometry separate from routing

The CSV rows are used to compute where the shuttle is on a segment. They do not decide which segment comes next. That decision comes only from `rail_network.yaml`. This separation prevents a common bug: two segment endpoints can be physically connected but not numerically identical. If routing used exact endpoint equality, tiny measurement or export errors could break the graph.

The CSV geometry answers this question:

```
At distance s along segment A23, where is the shuttle pose?
```

The topology answers this question:

```
When A23 ends, which segment is valid under the current A3 switch state?
```

Step 3: preprocess and normalize the CSV files

The preprocessing step reads the raw segment CSV files, removes duplicate consecutive points, preserves the directed point order, and writes normalized CSV files. Duplicate points are removed because a zero-length edge has no useful direction and can create unstable yaw values.

The preprocessing command is:

```
ros2 run mfja_robot_control_config room_315_csv_preprocessor.py
```

The relevant inputs and outputs are:

- raw CSV input: `mfja_robot_control_config/config/room_315_kinematics/raw_segments`;
- normalized CSV output: `mfja_robot_control_config/config/room_315_kinematics/normalized_segments`;
- segment summary: `mfja_robot_control_config/config/room_315_kinematics/segment_summary.yaml`.

Step 4: create explicit node anchors

After the geometry exists, endpoints are matched to named anchors. The current network has twelve anchors:

```
A1_C A1_G A1_S
A2_C A2_G A2_S
A3_C A3_G A3_S
A4_C A4_G A4_S
```

The suffix meaning is:

- `C`: switch center or common connection point;
- `G`: endpoint on the `G` branch;
- `S`: endpoint on the `S` branch.

The validator snaps segment endpoints to these anchors using a tolerance of `0.05 m`. Snapping here means validation association, not moving the runtime shuttle by magic. The association says: this segment endpoint belongs to this named node if it is close enough.

Step 5: define the graph and switch behavior

After endpoints are attached to anchors, the YAML file defines how the directed graph behaves. Fixed connections are simple. Switches are explicit. Converging switches use guard logic so the shuttle fails if it approaches a branch that is not selected.

The three rule types are:

- `fixed`: always move to the same next segment;
- `switch_select`: choose the outgoing branch from the switch state;
- `switch_guard`: allow entry only if the switch state matches the incoming branch.

This is why the model is more precise than a simple "big loop/small loop" toggle. The current state of each switch affects only the future route when the shuttle reaches that switch. It does not teleport the shuttle or rewrite the segment it is already on.

Step 6: validate visually and numerically

The network validator checks the geometry and the topology together:

- segment lengths;
- removed duplicate counts;
- endpoint-to-anchor distances;
- gaps above `0.05 m`;
- tangent mismatches above `25 deg`;
- whole-network debug plots with segment and node labels.

The validation command is:

```
ros2 run mfja_robot_control_config room_315_network_validator.py
```

The expected result for the current data is:

```
Status: PASS (0 warnings)
```

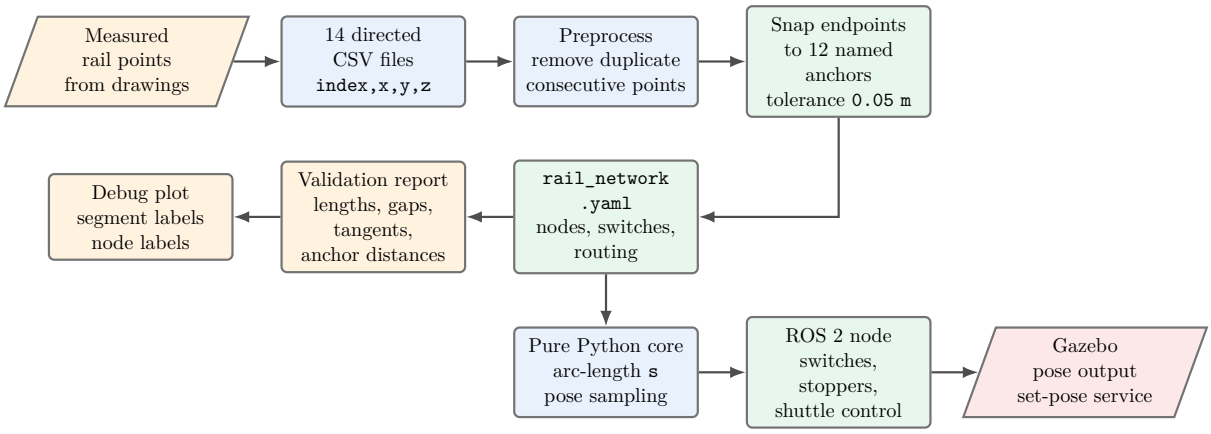

Figure 7.1: End-to-end flow chart for converting measured Room 315 rail data into a running kinematic shuttle path. CSV files provide geometry. The YAML file provides topology and routing. The ROS node combines both and publishes Gazebo poses.

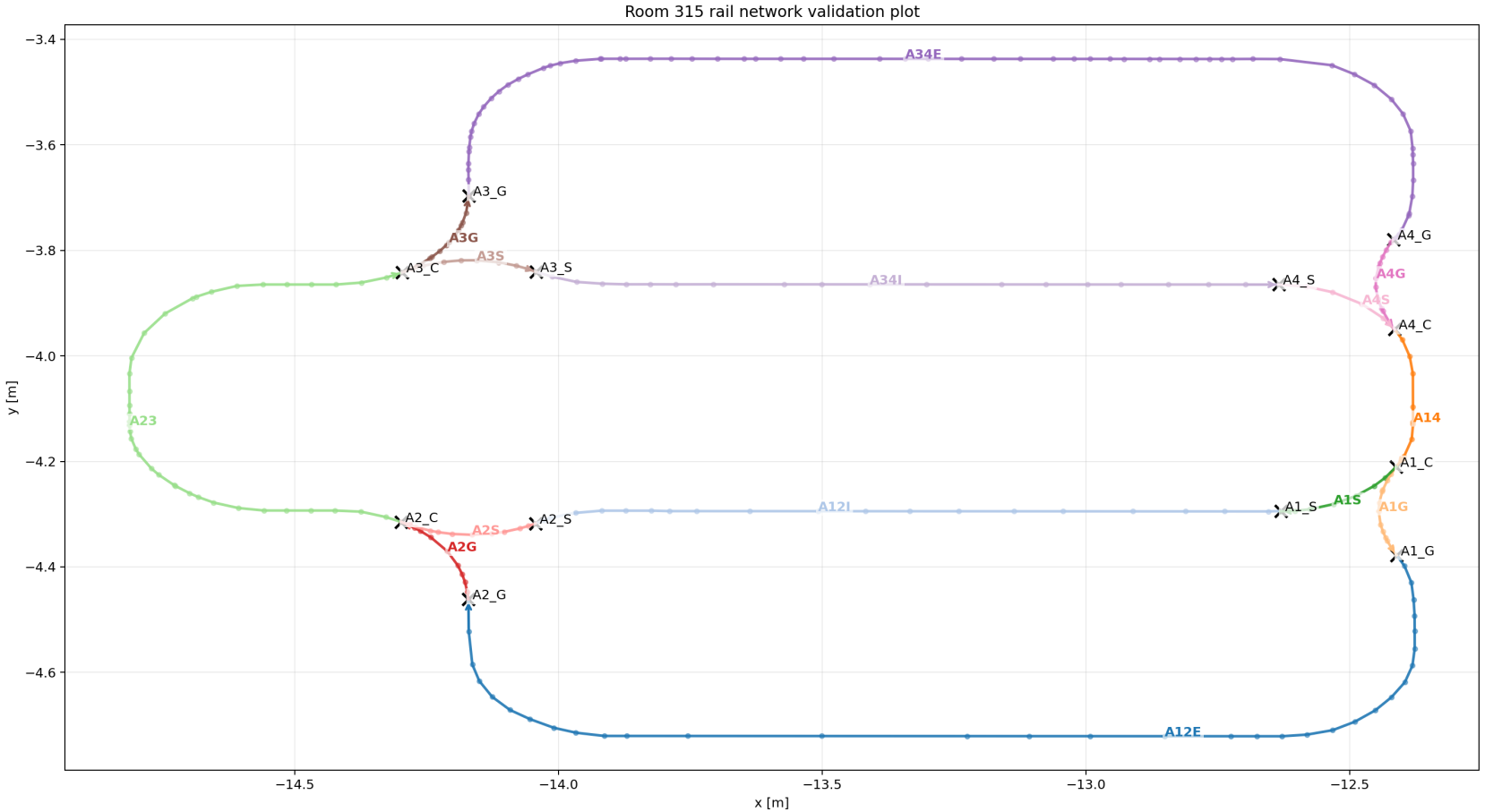

Figure 7.2: Room 315 directed rail network validation plot. Segment labels show the directed CSV segments. Node labels such as A1_C, A1_G, and A1_S show the explicit anchors used by the topology layer.

Step 7: use arc length during simulation

At runtime, the Python core converts each CSV polyline into cumulative arc-length values. If the shuttle state says `current_segment=A23` and `s=0.40`, the core samples the point located `0.40 m` along `A23`. Yaw is computed from the local CSV edge direction.

This gives the shuttle a continuous motion variable `s`, even though the path is stored as discrete CSV rows. The shuttle never jumps from CSV point to CSV point as a state machine. It interpolates along the polyline.

Step 8: apply the route only at segment boundaries

Switch states are checked when the shuttle reaches a segment end. Changing a switch while the shuttle is in the middle of a segment changes only the future successor. It does not move the shuttle to another branch immediately. This is the behavior needed for a realistic kinematic-first model.

For example:

```
Shuttle is on A23.
A3 changes from G to S.
Shuttle stays on A23.
When A23 ends, the routing table selects A3S instead of A3G.
```

Step 9: bake accepted calibration into the CSV data

During calibration, runtime scale and offset parameters were used to align the shuttle path with the visible rail in Gazebo. Once the correct values were found, the CSV coordinates were updated so normal operation no longer requires manual scale and offset parameters. This makes startup simpler and less error prone.

The normal node command should therefore use default pose scale and offset. Runtime calibration parameters remain available only for future measurement tests.

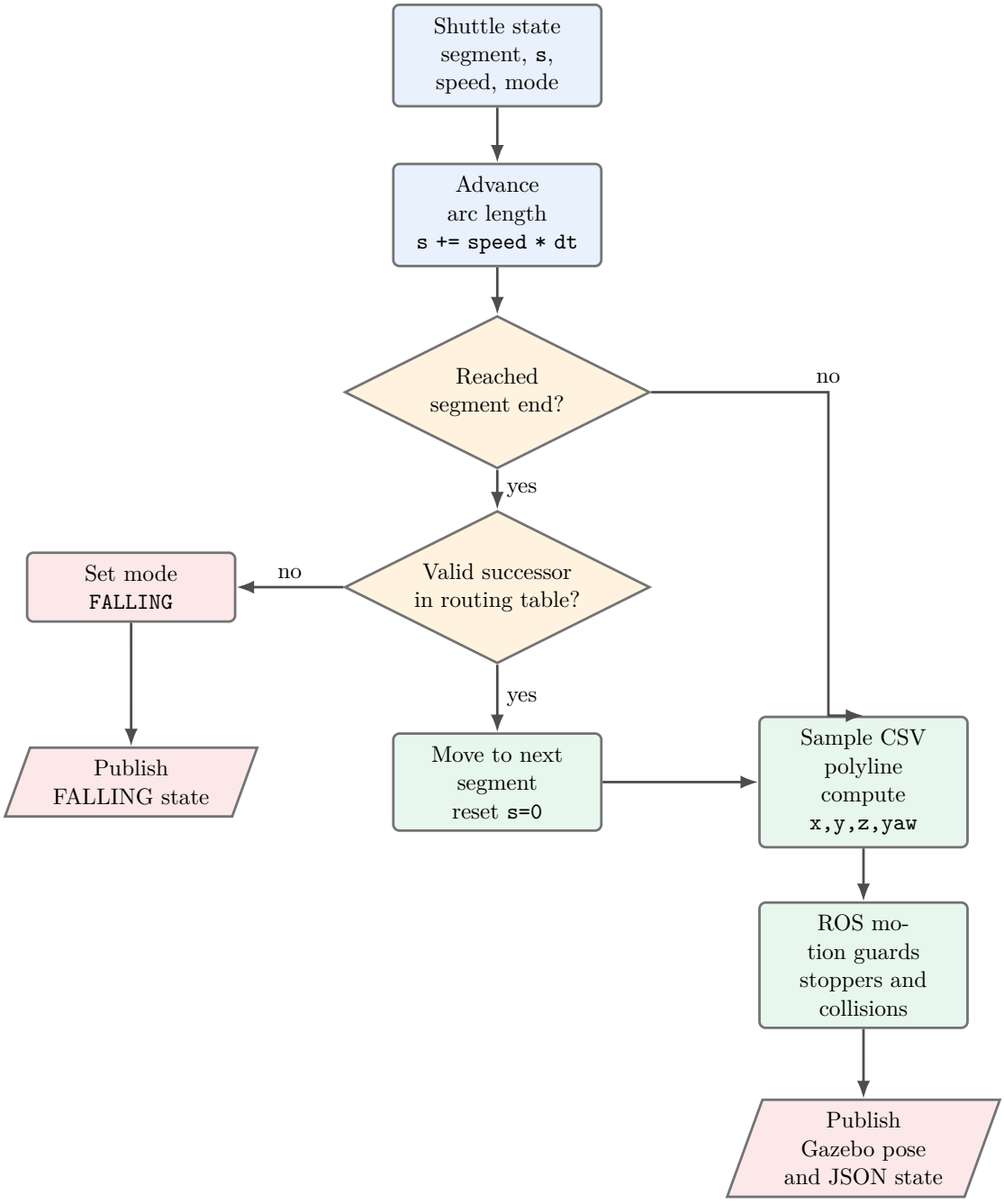

Figure 7.3: Runtime pose computation flow. The CSV file is sampled by arc length for geometry, while the routing table is consulted only when a segment ends.

```yaml
schema_version: 2
network_id: room_315_directed_one_way
description: 'Explicit directed rail topology for the room 315 shuttle network. Every
  segment is traversed forward-only, from CSV index 0 to the last row.

  '
source_csv_dir: mfja_robot_control_config/config/room_315_kinematics/raw_segments
normalized_csv_dir: mfja_robot_control_config/config/room_315_kinematics/normalized_segments
snap_tolerance_m: 0.05
motion_policy:
  segment_direction: forward_only
  traversal_rule: from_csv_index_0_to_last
  reverse_traversal_allowed: false
  on_missing_successor: FALLING
  topology_source_of_truth: explicit_nodes_and_routing_tables
switch_state_space:
  values:
  - G
  - S
  note: 'Switch states match the segment suffixes. G selects the G branch, and S selects
    the S branch. A later command layer may expose LEFT/RIGHT aliases if desired.

    '
stopper_state_space:
  values:
  - '0'
  - '1'
  note: 'Stopper states are independent from switch states. 0 means open/released,
    and 1 means stop/closed.

    '
```

Listing 7.2: Top-level Room 315 network policy and state spaces.

The relevant behavior encoded here is:

- all rail segments are directed and forward-only;
- a segment is traversed from CSV index `0` to the last CSV row;
- reverse traversal is explicitly disabled;
- if no valid successor exists, the shuttle enters `FALLING`;
- switch states are independent from stopper states;
- topology and routing tables are the source of truth.

This is why the system does not silently auto-correct a bad path. If a switch state makes the current rail branch invalid, the shuttle core reports a failure state. That failure is useful during validation because it exposes topology mistakes immediately instead of hiding them behind automatic rerouting.

7.3 Start slots and physical entry poses

The current system intentionally exposes only four valid start slots. These are the four entry locations that were accepted for the current Room 315 phase. The same values appear in both the YAML configuration and the Python fallback constants. The YAML version is the preferred configuration source. The Python constants keep the node usable if an older network file does not yet contain `start_slots`.

```
class AllowedStartPose:
    x: float
    y: float
    z: float
    roll: float
    pitch: float
    yaw: float


ALLOWED_START_POSES = {
    '1': AllowedStartPose(-14.95, -3.86, 0.84, 0.0, 0.0, 3.14),
    '2': AllowedStartPose(-15.43, -3.86, 0.84, 0.0, 0.0, 3.14),
    '3': AllowedStartPose(-15.24, -5.54, 0.84, 0.0, 0.0, 0.0),
    '4': AllowedStartPose(-14.77, -5.54, 0.84, 0.0, 0.0, 0.0),
}


@dataclass(frozen=True)
class StopPoint:
    segment: str
    stop_s: float
    sensor_distance_m: float


@dataclass(frozen=True)
class StopperConfig:
    name: str
    before_switch: str
    default_state: str
    stop_points: tuple[StopPoint, ...]


@dataclass
class ManagedShuttle:
    entity_name: str
    start_slot: str
    start_pose: AllowedStartPose
    start_snap_distance_m: float
    core: KinematicShuttleCore
    pose_publisher: object
    pending_set_pose: object | None = None
    pending_spawn: object | None = None
    last_gazebo_set_pose_time: object | None = None
    gazebo_spawned: bool = False
    blocked_by: str | None = None
    collision_distance_m: float | None = None
    enabled: bool = True
    stopped_by: str | None = None
    stopper_distance_m: float | None = None
    set_pose_warning_logged: bool = False
    spawn_failure_logged: bool = False
```

Listing 7.3: Start-slot fallback constants and per-shuttle runtime state.

Each start slot contains a Gazebo pose in the order:

```
x y z roll pitch yaw
```

The current four slots are:

Slot	x	y	z	roll	pitch	yaw
1	-14.95	-3.86	0.84	0.0	0.0	3.14
2	-15.43	-3.86	0.84	0.0	0.0	3.14
3	-15.24	-5.54	0.84	0.0	0.0	0.0
4	-14.77	-5.54	0.84	0.0	0.0	0.0

At startup, the node does not blindly put a shuttle on the pose and call it valid. It

snaps the requested start pose to the nearest point on the rail network. The log message includes the selected segment, arc-length value, and snap distance. A good startup line looks like this:

```
start_slot=2, start_segment=A34E, start_s=1.455, start_snap_distance=0.005 m
```

The snap distance is important. It tells the operator how far the configured Gazebo start pose is from the rail geometry. A small value means the slot is aligned with the rail. A large value means the configured slot should be reviewed.

7.4 ROS node parameter model

The main runtime node is `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py`. It is intentionally parameter-driven so the same executable can be used in the Room-only world and the full-floor world.

```
        self.declare_parameter('network_yaml', str(_default_network_path()))
        self.declare_parameter('shuttle_count', 1)
        self.declare_parameter('start_slot', 2)
        self.declare_parameter('start_slots', '')
        self.declare_parameter('start_snap_tolerance_m', 0.25)
        self.declare_parameter('initial_segment', 'A14')
        self.declare_parameter('initial_s', 0.0)
        self.declare_parameter('speed', 0.25)
        self.declare_parameter('update_rate_hz', 30.0)
        self.declare_parameter('enable_collision_avoidance', True)
        self.declare_parameter('shuttle_collision_distance_m', 0.33)
        self.declare_parameter('collision_search_iterations', 12)
        self.declare_parameter('pose_topic', '/room_315/shuttle/pose_cmd')
        self.declare_parameter('pose_topic_prefix', '/room_315/shuttles')
        self.declare_parameter('state_topic', '/room_315/shuttle/state')
        self.declare_parameter('add_shuttle_command_topic', '/room_315/shuttle/add_cmd')
        self.declare_parameter('shuttle_control_command_topic', '/room_315/shuttle/control_cmd')
        self.declare_parameter('switch_command_topic', '/room_315/switch_states')
        self.declare_parameter('stopper_command_topic', '/room_315/stopper_states')
        self.declare_parameter('sensor_state_topic', '/room_315/sensors/switch_approach')
        self.declare_parameter('pose_offset_command_topic', '/room_315/shuttle/pose_offset_cmd')
        self.declare_parameter('visual_switch_command_topic', '/mfja/conveyor/switch_cmd')
        self.declare_parameter('visual_switch_state_topic', '/mfja/conveyor/switch_states')
        self.declare_parameter('sync_from_visual_switch_states', True)
        self.declare_parameter('frame_id', 'world')
        self.declare_parameter('enable_gazebo_set_pose', False)
        self.declare_parameter('gazebo_world_name', 'room_315_only')
        self.declare_parameter('gazebo_set_pose_service', '')
        self.declare_parameter('enable_gazebo_spawn', True)
        self.declare_parameter('gazebo_spawn_service', '')
        self.declare_parameter('shuttle_model_sdf', str(_default_shuttle_model_sdf_path()))
        self.declare_parameter('preloaded_shuttle_count', 4)
        self.declare_parameter('reject_occupied_start_slots', True)
        self.declare_parameter('start_slot_occupancy_radius_m', 0.33)
        self.declare_parameter('gazebo_entity_name', 'room315_shuttle_1')
        self.declare_parameter('gazebo_entity_names', '')
        self.declare_parameter('gazebo_set_pose_rate_hz', 10.0)
        self.declare_parameter('publish_visual_switch_commands', True)
        self.declare_parameter('enable_gazebo_pose_transform', True)
        self.declare_parameter('pose_transform_a', -0.893249246800)
        self.declare_parameter('pose_transform_b', 0.005839516878)
        self.declare_parameter('pose_transform_tx', -26.921427375871)
        self.declare_parameter('pose_transform_c', 0.001889497475)
        self.declare_parameter('pose_transform_d', 1.308619216904)
        self.declare_parameter('pose_transform_ty', 0.666926143808)
        self.declare_parameter('pose_transform_z_offset', 0.0)
        self.declare_parameter('pose_transform_yaw_offset', 0.0)
        self.declare_parameter('pose_scale_x', 1.0)
        self.declare_parameter('pose_scale_y', 1.0)
        self.declare_parameter('pose_scale_origin_x', -15.855195431322)
        self.declare_parameter('pose_scale_origin_y', -4.525523413467)
        self.declare_parameter('pose_offset_x', 0.0)
        self.declare_parameter('pose_offset_y', 0.0)
        self.declare_parameter('pose_offset_z', 0.0)
```

Listing 7.4: Main ROS node parameters for the kinematic shuttle system.

The key parameter groups are:

- network parameters: `network_yaml`, `start_snap_tolerance_m`;
- shuttle initialization: `shuttle_count`, `start_slot`, `start_slots`, `gazebo_entity_name`, `gazebo_entity_names`;
- motion timing: `speed`, `update_rate_hz`, `gazebo_set_pose_rate_hz`;
- safety behavior: `enable_collision_avoidance`, `shuttle_collision_distance_m`, `reject_occupied_start_slots`;

- command topics: switch, stopper, shuttle add, shuttle ON/OFF, pose offset;
- visual integration topics for the physical switch meshes;
- Gazebo integration: world name, set-pose service, spawn service, and SDF path;
- pose transform and calibration parameters.

The default collision distance is `0.33 m`. It is enabled by default and does not need to be written in normal terminal commands. This value was chosen as the current practical stopping distance for the shuttle model in the kinematic simulation.

7.5 Loading start slots from the network file

The node loads allowed start slots from `rail_network.yaml`. If the YAML file has no `start_slots` section, the code falls back to the Python constants shown earlier.

```
421     def _load_allowed_start_poses(self) -> Dict[str, AllowedStartPose]:
422         raw_slots = self.network.config.get('start_slots') or {}
423         if not raw_slots:
424             return dict(ALLOWED_START_POSES)
425 
426         allowed: Dict[str, AllowedStartPose] = {}
427         for raw_slot, raw_config in raw_slots.items():
428             slot = str(raw_slot).strip()
429             pose_values = raw_config.get('pose') if isinstance(raw_config, dict) else raw_config
430             if pose_values is None or len(pose_values) != 6:
431                 raise ValueError(
432                     f'start_slots.{slot} must define pose: [x, y, z, roll, pitch, yaw].'
433                 )
434             allowed[slot] = AllowedStartPose(*[float(value) for value in pose_values])
435 
436         if not allowed:
437             raise ValueError('rail_network.yaml start_slots must not be empty.')
438         return allowed
```

Listing 7.5: Allowed start-slot loading from `rail_network.yaml`.

This keeps configuration ownership clear: the network file owns the operational start slots, and the node only provides defaults for compatibility.

7.6 Stopper configuration and stop-point generation

The stoppers are independent from switches. Each stopper is attached to the approach side of a switch. A stopper does not decide the route. It only decides whether an approaching shuttle must stop before the switch.

```yaml
stopper_state_space:
  values:
  - '0'
  - '1'
  note: 'Stopper states are independent from switch states. 0 means open/released,
    and 1 means stop/closed.

    '
stoppers:
  A1:
    before_switch: A1
    default_state: '0'
    stop_offset_m: 0.08
    sensor_distance_m: 0.25
    segments:
    - A14
  A2:
    before_switch: A2
    default_state: '0'
    stop_offset_m: 0.08
    sensor_distance_m: 0.25
    segments:
    - A12E
    - A12I
  A3:
    before_switch: A3
    default_state: '0'
    stop_offset_m: 0.08
    sensor_distance_m: 0.25
    segments:
    - A23
  A4:
    before_switch: A4
    default_state: '0'
    stop_offset_m: 0.08
    sensor_distance_m: 0.25
    segments:
    - A34E
    - A34I
```

Listing 7.6: Stopper definitions in the Room 315 network file.

The YAML fields mean:

- `before_switch`: the switch protected by this stopper;
- `default_state`: `0` means released, `1` means closed;
- `stop_offset_m`: distance before the end of the approach segment;
- `sensor_distance_m`: distance window where sensor events are published;
- `segments`: approach segments where the stopper can stop a shuttle.

The node converts each configured segment into a concrete stop point. If the YAML does not specify an absolute arc-length `s`, the node computes:

```
stop_s = segment.length - stop_offset_m
```

```
    def _load_stopper_configs(self) -> Dict[str, StopperConfig]:
        configs: Dict[str, StopperConfig] = {}
        raw_configs = self.network.config.get('stoppers', {}) or {}
        for raw_name, raw_config in raw_configs.items():
            name = str(raw_name).strip().upper()
            before_switch = str(raw_config.get('before_switch', name)).strip().upper()
            default_state = self._normalize_stopper_state(
                str(raw_config.get('default_state', '0'))
            )
            default_sensor_distance_m = float(raw_config.get('sensor_distance_m', 0.25))
            default_stop_offset_m = float(raw_config.get('stop_offset_m', 0.08))
            stop_points: list[StopPoint] = []

            raw_stop_points = raw_config.get('stop_points')
            if raw_stop_points is None:
                raw_stop_points = [
                    {'segment': segment_name}
                    for segment_name in raw_config.get('segments', [])
                ]

            for raw_stop_point in raw_stop_points:
                segment_name = str(raw_stop_point['segment']).strip()
                if segment_name not in self.network.segments:
                    raise ValueError(
                        f'Stopper {name} references unknown segment {segment_name!r}.'
                    )

                segment = self.network.segments[segment_name]
                if 's' in raw_stop_point:
                    stop_s = float(raw_stop_point['s'])
                else:
                    stop_offset_m = float(
                        raw_stop_point.get('stop_offset_m', default_stop_offset_m)
                    )
                    stop_s = segment.length - stop_offset_m
                stop_s = max(0.0, min(stop_s, segment.length))
                sensor_distance_m = float(
                    raw_stop_point.get('sensor_distance_m', default_sensor_distance_m)
                )
                stop_points.append(
                    StopPoint(
                        segment=segment_name,
                        stop_s=stop_s,
                        sensor_distance_m=max(0.0, sensor_distance_m),
                    )
                )

            if not stop_points:
                raise ValueError(f'Stopper {name} must define at least one stop point.')
            configs[name] = StopperConfig(
                name=name,
                before_switch=before_switch,
                default_state=default_state,
                stop_points=tuple(stop_points),
            )
        return configs
```

Listing 7.7: Runtime loading of stopper definitions and stop-point conversion.

This keeps the YAML readable while still giving the motion core a precise arc-length stop location.

7.7 Switches, fixed transitions, and routing table

Switches are stored separately from fixed transitions. The switch section is human-readable: it explains which node is controlled and which outgoing branch belongs to each state.

```yaml
switches:
  A1:
    kind: diverging
    controlled_node: A1_C
    states:
    - G
    - S
    incoming_segments:
    - A14
    outgoing_by_state:
      G: A1G
      S: A1S
  A2:
    kind: converging_gate
    controlled_node: A2_C
    states:
    - G
    - S
    incoming_path_by_state:
      G:
        incoming_segment: A12E
        connector_segment: A2G
      S:
        incoming_segment: A12I
        connector_segment: A2S
    outgoing_segments:
    - A23
  A3:
    kind: diverging
    controlled_node: A3_C
    states:
    - G
    - S
    incoming_segments:
    - A23
    outgoing_by_state:
      G: A3G
      S: A3S
  A4:
    kind: converging_gate
    controlled_node: A4_C
    states:
    - G
    - S
    incoming_path_by_state:
      G:
        incoming_segment: A34E
        connector_segment: A4G
      S:
        incoming_segment: A34I
        connector_segment: A4S
    outgoing_segments:
    - A14
fixed_transitions:
  A1G: A12E
  A1S: A12I
  A2G: A23
  A2S: A23
  A3G: A34E
  A3S: A34I
  A4G: A14
  A4S: A14
```

Listing 7.8: Switch declarations and fixed transitions.

The runtime core does not infer route behavior from names. It reads the `routing_table`. The routing table is the executable graph policy.

```yaml
routing_table:
  A14:
    type: switch_select
    switch: A1
    by_state:
      G: A1G
      S: A1S
    on_unknown_state: FALLING
  A1G:
    type: fixed
    next_segment: A12E
  A1S:
    type: fixed
    next_segment: A12I
  A12E:
    type: switch_guard
    switch: A2
    by_state:
      G: A2G
      S: FALLING
    on_unknown_state: FALLING
  A12I:
    type: switch_guard
    switch: A2
    by_state:
      G: FALLING
      S: A2S
    on_unknown_state: FALLING
  A2G:
    type: fixed
    next_segment: A23
  A2S:
    type: fixed
```

Listing 7.9: Routing table excerpt showing select, guard, and fixed rules.

There are three important routing patterns:

- `fixed`: the segment always continues to one known successor;
- `switch_select`: the switch chooses the next branch;
- `switch_guard`: the shuttle may continue only if the switch state matches the branch it is physically approaching.

The guard behavior matters at converging switches. For example, if a shuttle is on `A12E`, it can enter `A2G` only when switch `A2` is in state `G`. If `A2` is in state `S`, the routing table returns `FALLING`. This represents an invalid physical configuration, not an automatic detour.

7.8 Pure Python kinematic core

The pure core is in `mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py`. It has no Gazebo dependency and no ROS dependency. This is deliberate: the network and one-shuttle logic can be tested offline before integrating with simulation services.

```
MOVING = 'MOVING'
WAITING = 'WAITING'
FALLING = 'FALLING'


@dataclass(frozen=True)
class Point3D:
    x: float
    y: float
    z: float

    def distance_to(self, other: 'Point3D') -> float:
        return math.dist((self.x, self.y, self.z), (other.x, other.y, other.z))


@dataclass
class ShuttleState:
    current_segment: str
    s: float
    speed: float
    mode: str = MOVING


@dataclass(frozen=True)
class ShuttlePose:
    x: float
    y: float
    z: float
    yaw: float
    current_segment: str
    s: float
    mode: str
```

Listing 7.10: Core shuttle state and pose data structures.

The shuttle state is intentionally small:

- `current_segment`: the active rail segment;
- `s`: progress along that segment in meters;
- `speed`: scalar speed in meters per second;
- `mode`: `MOVING`, `WAITING`, or `FALLING`.

The pose returned by the core includes the interpolated `x`, `y`, `z`, and yaw, plus the current segment and mode.

7.9 Arc-length interpolation

CSV rows are converted to a polyline. The core computes cumulative arc lengths along that polyline. During each update, it samples the point at distance `s`. This makes speed independent of CSV point spacing.

```
class SegmentGeometry:
    def __init__(self, name: str, points: Sequence[Point3D]) -> None:
        if len(points) < 2:
            raise ValueError(f'{name} must contain at least two points')
        self.name = name
        self.points = list(points)
        self.arc_lengths = self._compute_arc_lengths(self.points)
        if self.length <= 0.0:
            raise ValueError(f'{name} has zero total length')

    @staticmethod
    def _compute_arc_lengths(points: Sequence[Point3D]) -> List[float]:
        arc_lengths = [0.0]
        total = 0.0
        for previous, current in zip(points, points[1:]):
            total += previous.distance_to(current)
            arc_lengths.append(total)
        return arc_lengths

    @property
    def length(self) -> float:
        return self.arc_lengths[-1]

    def sample(self, s: float) -> tuple[Point3D, float]:
        clamped_s = max(0.0, min(s, self.length))
        for index in range(1, len(self.arc_lengths)):
            previous_s = self.arc_lengths[index - 1]
            current_s = self.arc_lengths[index]
            if clamped_s > current_s and index < len(self.arc_lengths) - 1:
                continue

            previous = self.points[index - 1]
            current = self.points[index]
            edge_length = current_s - previous_s
            ratio = 0.0 if edge_length == 0.0 else (clamped_s - previous_s) / edge_length
            x = previous.x + ratio * (current.x - previous.x)
            y = previous.y + ratio * (current.y - previous.y)
            z = previous.z + ratio * (current.z - previous.z)
            yaw = math.atan2(current.y - previous.y, current.x - previous.x)
            return Point3D(x=x, y=y, z=z), yaw

        previous = self.points[-2]
        current = self.points[-1]
        yaw = math.atan2(current.y - previous.y, current.x - previous.x)
        return current, yaw
```

Listing 7.11: Segment geometry, cumulative length, and pose sampling.

The current geometry is still polyline-based, so it follows the CSV points. The practical improvement already made was to use corrected CSV extraction and then bake the calibration scale and offset into the CSV coordinates. That is why normal node startup no longer needs `pose_scale_x`, `pose_scale_y`, `pose_offset_x`, or `pose_offset_y` for the accepted path alignment.

7.10 Successor resolution and explicit failure

The core asks `RailNetwork.resolve_successor()` what comes after the current segment. If there is no successor, the shuttle enters `FALLING`.

```
class RailNetwork:
    def __init__(self, network_path: Path, config: dict, segments: Dict[str, SegmentGeometry]) -> None:
        self.network_path = network_path
        self.config = config
        self.segments = segments
        self.routing_table = config.get('routing_table', {})
        self.switches = config.get('switches', {})
        self.valid_switch_states = set(config.get('switch_state_space', {}).get('values', []))

    @classmethod
    def from_yaml(cls, network_path: Path) -> 'RailNetwork':
        network_path = network_path.resolve()
        with network_path.open() as handle:
            config = yaml.safe_load(handle)

        segments = {
            name: SegmentGeometry(
                name=name,
                points=_read_csv_points(_resolve_path(segment_config['csv'], network_path)),
            )
            for name, segment_config in config['segments'].items()
        }
        return cls(network_path=network_path, config=config, segments=segments)

    def default_switch_states(self) -> Dict[str, str]:
        return {
            switch_name: switch_config.get('states', ['G'])[0]
            for switch_name, switch_config in self.switches.items()
        }

    def normalized_switch_state(self, raw_state: str) -> str:
        state = raw_state.strip().upper()
        if state not in self.valid_switch_states:
            raise ValueError(
                f'Unknown switch state {raw_state!r}; expected one of {sorted(self.valid_switch_states)}
 '
            )
        return state

    def resolve_successor(
        self,
        current_segment: str,
        switch_states: Dict[str, str],
    ) -> Optional[str]:
        rule = self.routing_table.get(current_segment)
        if rule is None:
            return None

        rule_type = rule.get('type')
        if rule_type == 'fixed':
            next_segment = rule.get('next_segment')
            return None if next_segment == FALLING else next_segment

        switch_name = rule.get('switch')
        switch_state = switch_states.get(switch_name)
        if switch_state is None:
            return None

        normalized_state = self.normalized_switch_state(switch_state)
        next_segment = rule.get('by_state', {}).get(normalized_state, rule.get('on_unknown_state'))
        if next_segment == FALLING:
            return None
        return next_segment
```

Listing 7.12: Rail network loading, switch normalization, and successor lookup.

The function treats `FALLING` as a deliberate result. This is safer than guessing a route because it tells the operator or developer that the current switch configuration cannot support the current segment.

7.11 Core stepping logic

The step loop advances by `speed * dt`. If the remaining distance reaches the end of the current segment, the core consumes that distance, resolves the successor, and continues on the next segment. One update can cross more than one short segment if the speed and time step allow it.

```
    def step(self, dt: float, switch_states: Optional[Dict[str, str]] = None) -> ShuttlePose:
        if dt < 0.0:
            raise ValueError('dt must be non-negative')

        switch_states = switch_states or self.network.default_switch_states()
        if self.state.mode == FALLING:
            return self.pose()

        if self.state.speed <= 0.0:
            self.state.mode = WAITING
            return self.pose()

        if self.state.mode == WAITING:
            self.state.mode = MOVING

        remaining_distance = self.state.speed * dt
        while remaining_distance > 0.0 and self.state.mode == MOVING:
            segment = self.network.segments[self.state.current_segment]
            distance_to_end = max(0.0, segment.length - self.state.s)

            if remaining_distance < distance_to_end:
                self.state.s += remaining_distance
                remaining_distance = 0.0
                break

            remaining_distance -= distance_to_end
            self.state.s = segment.length
            successor = self.network.resolve_successor(self.state.current_segment, switch_states)
            if successor is None or successor not in self.network.segments:
                self.state.mode = FALLING
                break

            self.state.current_segment = successor
            self.state.s = 0.0

        return self.pose()
```

Listing 7.13: Pure kinematic stepping and FALLING behavior.

The core does not know about stoppers, sensors, collision avoidance, Gazebo spawning, or visual switch meshes. Those are ROS-node responsibilities layered around the pure core.

7.12 Runtime shuttle creation

The node supports runtime shuttle addition through `/room_315/shuttle/add_cmd`. The command parser accepts plain slot numbers, assignment strings, and JSON payloads.

```
627     def _parse_add_shuttle_command(self, raw_command: str) -> tuple[str, str, float]:
628         command = raw_command.strip()
629         if not command:
630             raise ValueError('Empty add shuttle command')
631 
632         if command.startswith('{'):
633             payload = json.loads(command)
634             assignments = [(str(key), str(value)) for key, value in payload.items()]
635         elif '=' not in command and ':' not in command:
636             assignments = [('slot', command)]
637         else:
638             assignments = []
639             for token in re.split(r'[\s,;]+', command.replace(':', '=')):
640                 if not token:
641                     continue
642                 if '=' not in token:
643                     raise ValueError(
644                         f'Add shuttle command must look like slot=3, got {token!r}'
645                     )
646                 key, raw_value = token.split('=', 1)
647                 assignments.append((key, raw_value))
648 
649         slot = ''
650         entity_name = ''
651         speed = self.default_shuttle_speed
652         for raw_key, raw_value in assignments:
653             key = raw_key.strip().lower()
654             value = raw_value.strip()
655             if key in {'slot', 'start_slot', 'start'}:
656                 slot = self._normalize_start_slot(value)
657             elif key in {'entity', 'entity_name', 'gazebo_entity_name', 'name'}:
658                 entity_name = value
659             elif key == 'speed':
660                 speed = float(value)
661             else:
662                 raise ValueError(
663                     f'Unknown add shuttle key {raw_key!r}; use slot, entity, or speed.'
664                 )
665 
666         if not slot:
667             slot = self._next_unused_start_slot()
668         if self.reject_occupied_start_slots:
669             occupied_by = self._start_slot_occupancy_blocker(slot)
670             if occupied_by is not None:
671                 entity_name_at_slot, distance_m = occupied_by
672                 raise ValueError(
673                     f'start slot {slot} is occupied by {entity_name_at_slot} '
674                     f'at distance {distance_m:.3f} m; add command rejected.'
675                 )
676 
677         if not entity_name:
678             entity_name = self._next_unused_entity_name()
679         if any(shuttle.entity_name == entity_name for shuttle in self.shuttles):
680             raise ValueError(
681                 f'Gazebo entity {entity_name!r} is already controlled by this node.'
682             )
683 
684         return entity_name, slot, speed
685 
686     def _next_unused_start_slot(self) -> str:
687         slots = sorted(self.allowed_start_poses)
688         if not self.reject_occupied_start_slots:
689             return slots[len(self.shuttles) % len(slots)]
690 
691         for slot in slots:
692             if self._start_slot_occupancy_blocker(slot) is None:
693                 return slot
694         raise ValueError('All allowed start slots are currently occupied.')
695 
696     def _start_slot_occupancy_blocker(self, slot: str) -> tuple[str, float] | None:
697         start_pose = self.allowed_start_poses[slot]
698         blockers = []
```

```
        for shuttle in self.shuttles:
            pose = self._to_gazebo_pose(shuttle.core.pose())
            distance_m = math.hypot(start_pose.x - pose.x, start_pose.y - pose.y)
            if distance_m < self.start_slot_occupancy_radius_m:
                blockers.append((shuttle.entity_name, distance_m))
        if not blockers:
            return None
        return sorted(blockers, key=lambda item: item[1])[0]
```

Listing 7.14: Runtime add-shuttle parser and occupied-slot rejection.

The current default behavior is strict: if the requested start slot is occupied inside the configured occupancy radius, the add request is rejected. This matches the meeting requirement that adding a shuttle into an occupied start location should say no instead of silently placing another shuttle there.

Examples of valid add commands are:

```
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=3'}"
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'entity=room315_shuttle_5
    slot=4 speed=0.2'}"
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: '{\"slot\":\"2\",\"entity
    \":\"room315_shuttle_6\",\"speed\":\"0.15\"}'}"
```

Expected rejection examples are:

```
# Rejected if a shuttle is still inside the slot-1 occupancy radius.
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=1'}"

# Rejected because only slots 1, 2, 3, and 4 exist.
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=5'}"
```

7.13 Stopper command implementation

The stopper topic is `/room_315/stopper_states`. The command is intentionally simple: `0` means open or released, and `1` means closed or stop.

```
    def _on_stopper_command(self, message: String) -> None:
        try:
            updates = self._parse_stopper_command(message.data)
        except (ValueError, json.JSONDecodeError) as error:
            self.get_logger().error(str(error))
            return

        self.stopper_states.update(updates)
        self.get_logger().info(f'Updated stopper states: {self.stopper_states}')

    def _parse_stopper_command(self, raw_command: str) -> Dict[str, str]:
        assignments = self._parse_assignments(raw_command, 'Stopper')
        updates: Dict[str, str] = {}
        for raw_selector, raw_state in assignments:
            selector = raw_selector.strip().upper()
            state = self._normalize_stopper_state(raw_state)
            if selector == 'ALL':
                for stopper_name in self.stopper_configs:
                    updates[stopper_name] = state
                continue
            if selector not in self.stopper_configs:
                allowed = ', '.join(['ALL', *sorted(self.stopper_configs)])
                raise ValueError(
                    f'Unknown stopper selector {selector!r}; use one of: {allowed}.'
                )
            updates[selector] = state
        return updates

    @staticmethod
    def _normalize_stopper_state(raw_state: str) -> str:
        state = str(raw_state).strip().upper()
        if state in {'1', 'ON', 'STOP', 'STOPPED', 'CLOSED', 'BLOCK', 'BLOCKED', 'TRUE'}:
            return '1'
        if state in {'0', 'OFF', 'OPEN', 'RELEASE', 'UNSTOP', 'UNBLOCK', 'FALSE'}:
            return '0'
        raise ValueError(
            f'Unknown stopper state {raw_state!r}; use 1/STOP/CLOSED or 0/OPEN/RELEASE.'
        )
```

Listing 7.15: Stopper command parser and accepted aliases.

The accepted stopper aliases are useful during manual testing:

- closed state: `1`, `ON`, `STOP`, `CLOSED`, `BLOCK`, `TRUE`;
- open state: `0`, `OFF`, `OPEN`, `RELEASE`, `UNSTOP`, `FALSE`;
- selector: `A1`, `A2`, `A3`, `A4`, or `ALL`.

Example commands:

```
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=1'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=0'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A2=STOP A3=OPEN'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'ALL=RELEASE'}"
```

7.14 Per-shuttle ON/OFF implementation

The shuttle ON/OFF topic is `/room_315/shuttle/control_cmd`. It controls movement permission for each shuttle independently. Turning a shuttle OFF does not delete it from Gazebo. It leaves the shuttle controlled by the node, sets the mode to `WAITING`, and marks `stopped_by` as `DISABLED`.

```
    def _on_shuttle_control_command(self, message: String) -> None:
        try:
            updates = self._parse_shuttle_control_command(message.data)
        except (ValueError, json.JSONDecodeError) as error:
            self.get_logger().error(str(error))
            return

        for entity_name, enabled in updates.items():
            shuttle = self._find_shuttle(entity_name)
            if shuttle is None:
                self.get_logger().error(
                    f'Unknown shuttle {entity_name!r}; command ignored.'
                )
                continue
            shuttle.enabled = enabled
            if not enabled:
                shuttle.core.state.mode = WAITING
                shuttle.stopped_by = 'DISABLED'
                shuttle.stopper_distance_m = 0.0
            else:
                if shuttle.stopped_by == 'DISABLED':
                    shuttle.stopped_by = None
                    shuttle.stopper_distance_m = None
                if shuttle.core.state.mode == WAITING and shuttle.core.state.speed > 0.0:
                    shuttle.core.state.mode = MOVING
        self.get_logger().info(
            'Updated shuttle enable states: '
            f'{ {shuttle.entity_name: shuttle.enabled for shuttle in self.shuttles} }'
        )

    def _parse_shuttle_control_command(self, raw_command: str) -> Dict[str, bool]:
        assignments = self._parse_assignments(raw_command, 'Shuttle control')
        keyed_payload = {key.strip().lower(): value for key, value in assignments}
        if {'entity', 'entity_name', 'name', 'shuttle'} & set(keyed_payload):
            entity_name = (
                keyed_payload.get('entity')
                or keyed_payload.get('entity_name')
                or keyed_payload.get('name')
                or keyed_payload.get('shuttle')
            )
            raw_state = (
                keyed_payload.get('enabled')
                or keyed_payload.get('state')
                or keyed_payload.get('mode')
                or keyed_payload.get('power')
            )
            if entity_name is None or raw_state is None:
                raise ValueError(
                    'Shuttle control command with entity=... must also include enabled=ON/OFF.'
                )
            return {entity_name: self._normalize_enabled_state(raw_state)}

        updates: Dict[str, bool] = {}
        for raw_selector, raw_state in assignments:
            selector = raw_selector.strip()
            enabled = self._normalize_enabled_state(raw_state)
            if selector.upper() == 'ALL':
                for shuttle in self.shuttles:
                    updates[shuttle.entity_name] = enabled
            else:
                updates[selector] = enabled
        return updates

    @staticmethod
    def _normalize_enabled_state(raw_state: str) -> bool:
        state = str(raw_state).strip().upper()
        if state in {'1', 'ON', 'ENABLE', 'ENABLED', 'START', 'RUN', 'TRUE'}:
            return True
        if state in {'0', 'OFF', 'DISABLE', 'DISABLED', 'STOP', 'PAUSE', 'FALSE'}:
            return False
        raise ValueError(
            f'Unknown shuttle control state {raw_state!r}; use ON/OFF or ENABLE/DISABLE.'
```

```
        )
```

Listing 7.16: Per-shuttle ON/OFF control implementation.

Accepted ON aliases include `1`, `ON`, `ENABLE`, `START`, `RUN`, and `TRUE`. Accepted OFF aliases include `0`, `OFF`, `DISABLE`, `STOP`, `PAUSE`, and `FALSE`.

Example commands:

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=OFF
    '}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=ON'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=ON'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: '{\"entity\":\"
    room315_shuttle_3\",\"enabled\":\"OFF\"}'}"
```

7.15 Motion guard order

The ROS node wraps the pure kinematic core with runtime motion guards. The order is:

1. if the shuttle is disabled, keep it waiting;
2. if an active stopper is ahead, limit the step so the shuttle stops at the stop point;
3. apply collision avoidance against already occupied shuttle poses;
4. if neither guard blocks the motion, let the pure kinematic core advance.

```
    def _step_with_motion_guards(
        self,
        shuttle: ManagedShuttle,
        dt: float,
        occupied_poses: Dict[str, ShuttlePose],
    ) -> ShuttlePose:
        if not shuttle.enabled:
            shuttle.core.state.mode = WAITING
            shuttle.blocked_by = None
            shuttle.collision_distance_m = None
            shuttle.stopped_by = 'DISABLED'
            shuttle.stopper_distance_m = 0.0
            return shuttle.core.pose()

        shuttle.stopped_by = None
        shuttle.stopper_distance_m = None
        active_stop = self._active_stopper_ahead(shuttle)
        effective_dt = dt
        stop_reached = False
        if active_stop is not None and shuttle.core.state.speed > 0.0:
            stopper_name, stop_point, distance_m = active_stop
            if distance_m <= 1e-6:
                shuttle.core.state.s = stop_point.stop_s
                shuttle.core.state.mode = WAITING
                shuttle.blocked_by = None
                shuttle.collision_distance_m = None
                shuttle.stopped_by = stopper_name
                shuttle.stopper_distance_m = 0.0
                return shuttle.core.pose()

            time_to_stop = distance_m / shuttle.core.state.speed
            if time_to_stop <= dt:
                effective_dt = time_to_stop
                stop_reached = True

        pose = self._step_with_collision_avoidance(
            shuttle=shuttle,
            dt=effective_dt,
            occupied_poses=occupied_poses,
        )
        if shuttle.blocked_by is not None or pose.mode == FALLING:
            return pose

        if stop_reached and active_stop is not None:
            stopper_name, stop_point, _distance_m = active_stop
            if shuttle.core.state.current_segment == stop_point.segment:
                shuttle.core.state.s = stop_point.stop_s
            shuttle.core.state.mode = WAITING
            shuttle.stopped_by = stopper_name
            shuttle.stopper_distance_m = 0.0
            return shuttle.core.pose()

        shuttle.stopped_by = None
        shuttle.stopper_distance_m = None
        return pose

    def _active_stopper_ahead(
        self,
        shuttle: ManagedShuttle,
    ) -> tuple[str, StopPoint, float] | None:
        state = shuttle.core.state
        candidates: list[tuple[str, StopPoint, float]] = []
        for stopper_name, stopper_config in self.stopper_configs.items():
            if self.stopper_states.get(stopper_name, '0') != '1':
                continue
            for stop_point in stopper_config.stop_points:
                if stop_point.segment != state.current_segment:
                    continue
                distance_m = stop_point.stop_s - state.s
                if distance_m >= -1e-6:
                    candidates.append((stopper_name, stop_point, max(0.0, distance_m)))
        if not candidates:
```

```
            return None
        return sorted(candidates, key=lambda item: item[2])[0]
```

Listing 7.17: Motion guard wrapper for disabled shuttles and active stoppers.

This order matters. A disabled shuttle should not move even if the path is clear. A closed stopper should stop a shuttle before the switch. Collision avoidance then prevents two active shuttles from merging into each other.

7.16 Collision avoidance behavior

Collision avoidance uses center-distance checks between the Gazebo-space poses of shuttles. The default threshold is `0.33 m`. If the proposed full time step would collide, the code performs a binary search over the time step and moves only to the last safe position, then enters `WAITING`.

```
    def _step_with_collision_avoidance(
        self,
        shuttle: ManagedShuttle,
        dt: float,
        occupied_poses: Dict[str, ShuttlePose],
    ) -> ShuttlePose:
        if not self.enable_collision_avoidance or dt <= 0.0:
            shuttle.blocked_by = None
            shuttle.collision_distance_m = None
            return shuttle.core.step(dt, switch_states=self.switch_states)

        start_state = self._snapshot_shuttle_state(shuttle)
        start_pose = shuttle.core.pose()
        start_gazebo_pose = self._to_gazebo_pose(start_pose)
        blockers = self._collision_blockers(shuttle.entity_name, start_gazebo_pose, occupied_poses)
        if blockers:
            blocker_name, distance = blockers[0]
            self._restore_shuttle_state(shuttle, start_state)
            shuttle.core.state.mode = WAITING
            shuttle.blocked_by = blocker_name
            shuttle.collision_distance_m = distance
            return shuttle.core.pose()

        self._restore_shuttle_state(shuttle, start_state)
        proposed_pose = shuttle.core.step(dt, switch_states=self.switch_states)
        proposed_gazebo_pose = self._to_gazebo_pose(proposed_pose)
        blockers = self._collision_blockers(
            shuttle.entity_name,
            proposed_gazebo_pose,
            occupied_poses,
        )
        if not blockers:
            shuttle.blocked_by = None
            shuttle.collision_distance_m = None
            return proposed_pose

        high = dt
        low = 0.0
        blocker_name, distance = blockers[0]
        for _ in range(max(1, self.collision_search_iterations)):
            mid = 0.5 * (low + high)
            self._restore_shuttle_state(shuttle, start_state)
            mid_pose = shuttle.core.step(mid, switch_states=self.switch_states)
            mid_gazebo_pose = self._to_gazebo_pose(mid_pose)
            mid_blockers = self._collision_blockers(
                shuttle.entity_name,
                mid_gazebo_pose,
                occupied_poses,
            )
            if mid_blockers:
                high = mid
```

```
                blocker_name, distance = mid_blockers[0]
            else:
                low = mid

        self._restore_shuttle_state(shuttle, start_state)
        shuttle.core.step(low, switch_states=self.switch_states)
        shuttle.core.state.mode = WAITING
        shuttle.blocked_by = blocker_name
        shuttle.collision_distance_m = distance
        return shuttle.core.pose()
```

Listing 7.18: Collision avoidance with binary search for the last safe step.

This is a kinematic approximation, not full contact physics. It is intentionally simple for the current phase. Its purpose is to prevent visual overlap and to model the practical rule: if a shuttle reaches another shuttle, it waits instead of passing through it.

7.17 Sensor event publication

The sensor interface is educational and workflow-oriented. It publishes approach events whenever a shuttle is within the configured sensor distance before a stopper.

```
    def _sensor_events(self) -> list[dict]:
        events = []
        for shuttle in self.shuttles:
            state = shuttle.core.state
            for stopper_name, stopper_config in self.stopper_configs.items():
                for stop_point in stopper_config.stop_points:
                    if stop_point.segment != state.current_segment:
                        continue
                    distance_m = stop_point.stop_s - state.s
                    if 0.0 <= distance_m <= stop_point.sensor_distance_m:
                        events.append(
                            {
                                'sensor': f'{stopper_name}_APPROACH',
                                'stopper': stopper_name,
                                'before_switch': stopper_config.before_switch,
                                'entity_name': shuttle.entity_name,
                                'segment': state.current_segment,
                                'distance_m': distance_m,
                                'stopper_state': self.stopper_states.get(stopper_name, '0'),
                                'workflow': 'sensor -> stop shuttle -> move switch -> unstop shuttle',
                            }
                        )
        return events

    def _publish_sensor_state(self) -> None:
        message = String()
        message.data = json.dumps(
            {
                'sensors': self._sensor_events(),
                'stopper_states': self.stopper_states,
            },
            sort_keys=True,
        )
        self.sensor_state_publisher.publish(message)
```

Listing 7.19: Switch-approach sensor event generation.

A sensor event contains:

- `sensor`: for example `A3_APPROACH`;
- `stopper`: the stopper that can stop the shuttle;
- `before_switch`: the protected switch;

- `entity_name`: which shuttle triggered the event;
- `segment`: which segment the shuttle is currently on;
- `distance_m`: remaining distance to the stop point;
- `stopper_state`: whether the stopper is currently open or closed;
- `workflow`: the intended sequence, `sensor -> stop shuttle -> move switch -> unstop shuttle`.

When four shuttles are running, the sensor topic can contain events from any of them. The event is not anonymous. The `entity_name` field tells exactly which shuttle the event belongs to.

7.18 Published state payload

The state topic is `/room_315/shuttle/state`. It publishes a JSON string with global switch and stopper states plus a list of shuttles. Each shuttle entry includes the raw rail pose, transformed Gazebo pose, segment, arc length, mode, enabled flag, blocker name, stopper name, start slot, and speed.

```
    def _publish_state(
        self,
        raw_poses: list[ShuttlePose],
        gazebo_poses: list[ShuttlePose],
    ) -> None:
        shuttles_payload = []
        for shuttle, pose, gazebo_pose in zip(self.shuttles, raw_poses, gazebo_poses):
            shuttles_payload.append(
                {
                    **asdict(pose),
                    'entity_name': shuttle.entity_name,
                    'blocked_by': shuttle.blocked_by,
                    'collision_distance_m': shuttle.collision_distance_m,
                    'enabled': shuttle.enabled,
                    'stopped_by': shuttle.stopped_by,
                    'stopper_distance_m': shuttle.stopper_distance_m,
                    'gazebo_pose': asdict(gazebo_pose),
                    'start_slot': shuttle.start_slot,
                    'start_snap_distance_m': shuttle.start_snap_distance_m,
                    'speed': shuttle.core.state.speed,
                }
            )
```

Listing 7.20: Per-shuttle state fields published on the state topic.

This topic is the best first debug topic because it tells whether a shuttle is stopped by an OFF command, stopped by a stopper, blocked by another shuttle, or in `FALLING`.

Chapter 8

Complete Feature Usage Manual

This chapter lists the practical commands for every current feature. The commands are intentionally copy-and-paste friendly.

8.1 Terminal setup used by all examples

Every terminal should start with:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

If the workspace was modified, rebuild first:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash

colcon build --base-paths \
  src/mfja_3rd_floor_gz/mfja_robot_control_config \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_description \
  src/mfja_3rd_floor_gz/mfja_room_315_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_gz \
  --packages-select \
  mfja_robot_control_config \
  mfja_3rd_floor_description \
  mfja_room_315_bringup \
  mfja_3rd_floor_bringup \
  mfja_3rd_floor_gz \
  --symlink-install

source install/setup.bash
```

8.2 Complete feature checklist

The current repository contains both the older multi-robot simulation work and the newer Room 315 kinematic shuttle work. The table below maps each feature to the practical section that explains how to run it.

Feature	Where it is operated from
Build and source the workspace	Terminal setup section in this chapter.

Feature	Where it is operated from
Launch Room 315 only	`mfja_room_315_bringup room_315_only.launch.py`.
Launch the full third floor	`mfja_3rd_floor_bringup full_floor.launch.py`.
Spawn no robots, selected robots, or all robots	The `robots:=...` launch argument.
Control industrial robot arms	Namespaced `/robot_name/joint_trajectory` topics.
Control the TIAGo mobile base	The `/tiago1/cmd_vel` topic.
Run synchronized robot motions	`multi_robot_sync_demo.py`.
Run one or more Room 315 shuttles	`room_315_kinematic_shuttle_node.py`.
Add shuttles during simulation	`/room_315/shuttle/add_cmd`.
Turn shuttles ON or OFF	`/room_315/shuttle/control_cmd`.
Change route switches	`/room_315/switch_states`.
Stop and release shuttles before switches	`/room_315/stopper_states`.
Read educational sensor events	`/room_315/sensors/switch_approach`.
Inspect shuttle state and collision holds	`/room_315/shuttle/state`.
Validate CSV paths and the rail network	`room_315_csv_preprocessor.py`, `room_315_network_validator.py`, and `room_315_segment_plot.py`.

8.3 Terminal 1: run Room 315 only

Use this when testing Room 315 in isolation:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=none \
  start_paused:=false
```

If Gazebo opens paused, press the play button or restart with `start_paused:=false`.

8.4 Terminal 1 alternative: run the full floor

Use this when testing Room 315 inside the complete third-floor world:

```
cd /home/tiago/ALI_ros2_ws
```

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=none \
  start_paused:=false
```

The shuttle node must use the same Gazebo world name as the running world. For Room-only, use `room_315_only`. For the full floor, use `mfja_3rd_floor`.

8.5 Terminal 1 robot launch options

The launch argument `robots:=...` controls which robot models are spawned with the world. It supports:

- `robots:=none`: start the world without robots, useful for shuttle-only testing;
- no `robots` argument or `robots:=`: use the YAML `enabled` flags;
- `robots:=all`: spawn every robot entry from the selected YAML file;
- full names such as `kuka1` and `tiago1`;
- shortcuts such as `kuka`, `staubli`, `hc10`, `hc10dt`, and `tiago`;
- numeric indices by YAML order such as `1,2,5`.

Room 315 with all configured robot entries:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=all \
  start_paused:=false
```

Room 315 with selected robots only:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=kuka1,staubli1,yaskawa_hc10_1 \
  start_paused:=false
```

Full floor with all enabled robots from `robots.yaml`:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  start_paused:=false
```

Full floor with all configured robots, even if a YAML entry is disabled:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=all \
  start_paused:=false
```

Full floor with selected robots by shortcuts:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=kuka,staubli,hc10,hc10dt,tiago \
  start_paused:=false
```

8.6 Robot terminal A: verify robot topics

Open a new terminal after the world is running:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic list | grep -E '^/(kuka1|staubli1|yaskawa_hc10_1|yaskawa_hc10dt_1|tiago1)/'

ros2 topic info /kuka1/joint_trajectory
ros2 topic info /staubli1/joint_trajectory
ros2 topic info /yaskawa_hc10_1/joint_trajectory
ros2 topic info /yaskawa_hc10dt_1/joint_trajectory
ros2 topic info /tiago1/joint_trajectory
ros2 topic info /tiago1/cmd_vel
```

The joint trajectory topics are ROS-to-Gazebo command topics. The joint state, odometry, and progress topics can be inspected while the robots move:

```
ros2 topic echo /kuka1/joint_states
ros2 topic echo /staubli1/joint_states
ros2 topic echo /yaskawa_hc10_1/joint_states
ros2 topic echo /yaskawa_hc10dt_1/joint_states
ros2 topic echo /tiago1/odom
```

8.7 Robot terminal B: command individual robot arms

KUKA KR6 R900 sixx:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /kuka1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_a1','joint_a2','joint_a3','joint_a4','joint_a5','joint_a6'], points: [{positions:
      [0.6,-1.0,1.1,0.0,0.6,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

Staeubli TX2-60L:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /staubli1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1','joint_2','joint_3','joint_4','joint_5','joint_6'], points: [{positions:
     [0.1,0.4,-0.6,0.0,0.5,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

Yaskawa HC10:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /yaskawa_hc10_1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1_s','joint_2_l','joint_3_u','joint_4_r','joint_5_b','joint_6_t'], points: [{
    positions: [0.0,-0.6,0.8,0.0,0.5,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

Yaskawa HC10DT:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /yaskawa_hc10dt_1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1_s','joint_2_l','joint_3_u','joint_4_r','joint_5_b','joint_6_t'], points: [{
    positions: [0.0,-0.5,0.7,0.0,0.4,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

TIAGo arm and head:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /tiago1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['torso_lift_joint','arm_1_joint','arm_2_joint','arm_3_joint','arm_4_joint','arm_5_joint
    ','arm_6_joint','arm_7_joint','head_1_joint','head_2_joint'], points: [{positions:
    [0.10,0.3,-0.5,-0.4,1.0,0.2,-0.2,0.1,0.2,-0.2], time_from_start: {sec: 4, nanosec: 0}}]}"
```

8.8 Robot terminal C: command the TIAGo base

Move the TIAGo base slowly:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub -r 20 /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.15, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.20}}"
```

Stop the TIAGo base from another terminal, or press Ctrl-C in the publishing terminal and send one zero command:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

8.9 Robot terminal D: synchronized multi-robot commands

List supported joints and default poses:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py --list-joints
```

Move every enabled robot to its default preset:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --trajectory-duration 4.0
```

Move only selected robots with explicit goals:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
  --goal staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
  --trajectory-duration 4.0
```

Move TIAGo arm and base together for a short demo:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
  --goal tiago1=0.10,0.3,-0.5,-0.4,1.0,0.2,-0.2,0.1,0.2,-0.2 \
  --tiago-base-linear 0.10 \
  --tiago-base-angular 0.15 \
  --tiago-base-duration 3.0 \
  --trajectory-duration 4.0
```

8.10 Terminal 2: bridge the Gazebo set-pose service

For Room-only:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export GZ_PARTITION=room315

ros2 run ros_gz_bridge parameter_bridge \
  /world/room_315_only/set_pose@ros_gz_interfaces/srv/SetEntityPose
```

For the full floor:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run ros_gz_bridge parameter_bridge \
  /world/mfja_3rd_floor/set_pose@ros_gz_interfaces/srv/SetEntityPose
```

Check that the service exists:

```
ros2 service list | grep set_pose
```

8.11 Terminal 3: run one shuttle

Room-only one-shuttle command:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

```
ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=room_315_only \
  -p start_slot:=2 \
  -p enable_gazebo_set_pose:=true \
  -p gazebo_entity_name:=room315_shuttle_1 \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

Full-floor one-shuttle command:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=mfja_3rd_floor \
  -p start_slot:=2 \
  -p enable_gazebo_set_pose:=true \
  -p gazebo_entity_name:=room315_shuttle_1 \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

8.12 Terminal 3: four-shuttle startup

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=room_315_only \
  -p shuttle_count:=4 \
  -p start_slots:=1,2,3,4 \
  -p gazebo_entity_names:=room315_shuttle_1,room315_shuttle_2,room315_shuttle_3,room315_shuttle_4 \
  -p enable_gazebo_set_pose:=true \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

If running in the full-floor world, change only:

```
-p gazebo_world_name:=mfja_3rd_floor
```

8.13 Terminal 4: inspect state continuously

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic echo /room_315/shuttle/state
```

Useful fields:

- `entity_name`: identifies which shuttle the row belongs to;
- `current_segment`: current rail segment;
- `s`: arc-length progress on that segment;
- `mode`: `MOVING`, `WAITING`, or `FALLING`;
- `enabled`: per-shuttle ON/OFF state;

- `stopped_by`: stopper name or `DISABLED`;
- `blocked_by`: another shuttle if collision avoidance is holding it;
- `start_slot`: original start slot used for that shuttle.

8.14 Terminal 5: inspect sensor events

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic echo /room_315/sensors/switch_approach
```

Example event:

```
data: '{"sensors": [{"before_switch": "A3", "distance_m": 0.214, "entity_name": "room315_shuttle_4", "
     segment": "A23", "sensor": "A3_APPROACH", "stopper": "A3", "stopper_state": "0", "workflow": "
     sensor -> stop shuttle -> move switch -> unstop shuttle"}], "stopper_states": {"A1": "0", "A2":
     "0", "A3": "0", "A4": "0"}}'
```

This means shuttle `room315_shuttle_4` is approaching switch `A3` on segment `A23`. The remaining distance to the A3 stop point is about `0.214 m`. The stopper is currently open because `stopper_state` is `0`.

8.15 Switch commands

Switches are controlled through:

```
/room_315/switch_states
```

Use these commands to test all switches:

```
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A2=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A2=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A4=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A4=S'}"
```

Set all switches at once:

```
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=S'}"
```

Set a mixed route in one command:

```
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=S A2=G A3=S A4=G'}"
```

The switch command topic updates both routing logic and visual switch commands. Do not command only `/mfja/conveyor/switch_cmd` if the shuttle route must change. That topic is visual-only from the routing point of view.

8.16 Stop-switch-release workflow

The intended teaching workflow is:

1. watch `/room_315/sensors/switch_approach`;
2. close the stopper before the switch;
3. wait until the shuttle is stopped;
4. move the switch;
5. open the stopper.

Example for switch `A3`:

```
# Stop before A3.
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=1'}"

# Move the switch while the shuttle is stopped.
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=S'}"

# Release the shuttle.
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=0'}"
```

The same pattern applies to `A1`, `A2`, and `A4`.

8.17 Add shuttles during simulation

After the node is already running, add a shuttle from a free slot:

```
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=3'}"
```

Add with a custom entity name and speed:

```
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'entity=room315_shuttle_5
    slot=4 speed=0.2'}"
```

If the requested slot is still occupied, the node rejects the command. This is expected behavior. Wait until the current shuttle has moved away from the start slot, or choose another free slot.

8.18 Turn individual shuttles ON or OFF

Turn one shuttle OFF:

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=OFF
    '}"
```

Turn it ON again:

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=ON'}"
```

Turn all shuttles OFF or ON:

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=ON'}"
```

The state topic should show `enabled=false` and `stopped_by=DISABLED` for a disabled shuttle.

8.19 Pose calibration controls

The accepted CSV coordinates already include the calibration that was found during testing. Normal operation should use default scale and offset values. However, runtime calibration is still available for future alignment tests.

Start with temporary X and Y correction:

```
ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=room_315_only \
  -p enable_gazebo_set_pose:=true \
  -p gazebo_entity_name:=room315_shuttle_1 \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0 \
  -p pose_scale_x:=1.0 \
  -p pose_scale_y:=1.0 \
  -p pose_offset_x:=0.0 \
  -p pose_offset_y:=0.0 \
  -p pose_offset_z:=0.0
```

Change offset while the node is running:

```
ros2 topic pub --once /room_315/shuttle/pose_offset_cmd std_msgs/msg/String "{data: 'x=-0.02 y=0.01 z
    =0.00'}"
```

This should be used only for calibration experiments. Once the correct values are known, the preferred method is to bake the correction into the CSV path data and run the node with default calibration parameters.

8.20 Validation commands

Run preprocessing and validation:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_csv_preprocessor.py
ros2 run mfja_robot_control_config room_315_network_validator.py
ros2 run mfja_robot_control_config room_315_segment_plot.py
```

Expected validator result:

```
Status: PASS (0 warnings)
```

Generated files:

- normalized CSV output: `mfja_robot_control_config/config/room_315_kinematics/normalized_segments`;
- segment summary: `mfja_robot_control_config/config/room_315_kinematics/segment_summary.yaml`;
- validation report: `mfja_robot_control_config/config/room_315_kinematics/validation_report.yaml`;
- network plot: `mfja_robot_control_config/config/room_315_kinematics/debug_plots/network_validation.png`.

8.21 Troubleshooting table

Symptom	Most likely cause and action
`Gazebo set_pose service is not ready yet`	The bridge is missing, the world name is wrong, or Gazebo is not fully ready. Run `ros2 service list | grep set_pose` and make sure the shuttle node uses the same `gazebo_world_name`.
Shuttle does not appear in the full-floor world	The node may still be using `room_315_only`. Start it with `-p gazebo_world_name:=mfja_3rd_floor`.
Switch moves visually but the route does not change	The command was likely sent to the visual topic only. Send route commands to `/room_315/switch_states`.
Shuttle stops at a switch approach	Check `stopped_by` on the state topic. If it is a stopper name, release that stopper with `/room_315/stopper_states`.
Shuttle is waiting with `stopped_by=DISABLED`	It was turned OFF by `/room_315/shuttle/control_cmd`. Send `room315_shuttle_N=ON`.
Shuttle is waiting with `blocked_by` set	Collision avoidance is active. The named shuttle is too close. Move, stop, or release shuttles so spacing becomes safe.
Add command is rejected because a slot is occupied	This is the current correct behavior. Wait until the slot is clear or choose another slot.
Add command for slot 5 is rejected	Only four start slots exist in the current accepted configuration: `1`, `2`, `3`, and `4`.
Shuttle enters `FALLING`	The current segment has no valid successor under the current switch state. Check the routing table and current switch states.

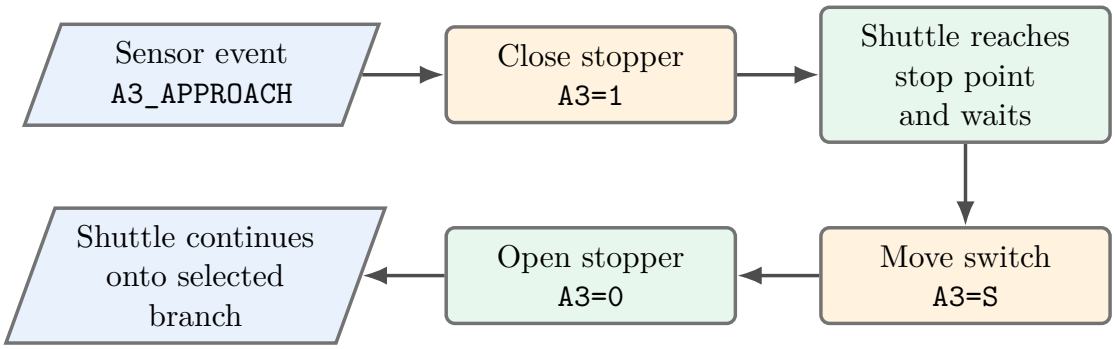

Figure 8.1: Stopper-based switch workflow. The stopper and switch are separate primitives: the stopper creates a safe waiting point, and the switch selects the future route.

Chapter 9

What Was Built in This Phase

This chapter summarizes the concrete engineering changes that define the current version of the repository.

9.1 Repository cleanup and package separation

The repository was cleaned into a meta-repository shape. Root-level package artifacts were removed or moved into the packages that own them. The current structure separates:

- Gazebo descriptions and worlds;
- launch entry points for Room 315 and the full floor;
- shared robot and shuttle control configuration;
- compatibility launch entry points;
- repository-level documentation.

This makes the project easier to build with explicit `colcon` package selection and prevents unrelated assets from being mixed at the root.

9.2 Dynamic shuttle work removed from the current main path

Earlier dynamic/contact-based shuttle experiments were removed from this branch because they were not the accepted direction for the current phase. The current approach is kinematic-first:

- validate CSV rail geometry offline;
- validate explicit topology and switch routing;
- run one-shuttle logic without Gazebo;
- add ROS and Gazebo pose control;
- add multiple shuttles, stoppers, sensors, and simple collision avoidance;

- postpone contact dynamics, wheels, and rail contact physics.

This order reduces risk because path logic can be tested before physics complexity is introduced.

9.3 Room 315 rail graph and validation pipeline

The rail network was split into directed CSV segments. The accepted segment names are:

```
A1G A1S A2G A2S A3G A3S A4G A4S A12E A12I A14 A23 A34E A34I
```

The validation tools produce:

- normalized CSV files;
- segment lengths;
- duplicate-removal counts;
- endpoint-to-anchor distances;
- gap warnings above `0.05 m`;
- tangent mismatch warnings above `25 deg`;
- a debug plot with node labels and segment labels.

The latest expected result is a clean pass with no warnings.

9.4 Switches made independent and visible

The shuttle route switch state is controlled by `/room_315/switch_states`. Commands sent there update both the graph routing state and the visual conveyor switch command topic. This gives one operator command for logical and visual behavior.

The current logical switch states are `G` and `S`, matching the segment suffixes in the network file. Historical aliases such as left/right or specific visual names may exist in the visual layer, but the current route graph uses `G/S` as the stable internal representation.

9.5 Stopper primitive added before switches

Independent binary stoppers were added before switches `A1`, `A2`, `A3`, and `A4`. This fills the workflow gap from the meeting requirement:

```
sensor -> stop shuttle -> move switch -> unstop shuttle
```

The stopper primitive is intentionally separate from the switch primitive. A stopper answers the question: should the shuttle wait before entering this switch area? A switch answers the question: which route is selected?

9.6 Sensor workflow added

The sensor topic publishes approach events before a switch. This is not a full hardware sensor simulation yet. It is a deterministic software sensor derived from the shuttle arc length and the configured stop point. It is enough for the teaching and testing workflow because it tells the operator:

- which shuttle is approaching;
- which switch is ahead;
- which stopper protects that switch;
- how far the shuttle is from the stop point;
- whether the stopper is currently open or closed.

9.7 Per-shuttle control added

Each shuttle can now be turned ON or OFF independently. This was added because the system needs an operational primitive that can pause a specific shuttle without stopping the entire node or deleting the model from Gazebo.

The OFF state is visible in the state payload as:

```
enabled: false
mode: WAITING
stopped_by: DISABLED
```

9.8 Runtime shuttle addition with occupied-slot rejection

The node can spawn or control additional shuttles during simulation. The current accepted policy rejects a new shuttle if the requested start slot is occupied. This differs from the earlier experimental behavior where repeated slot use was allowed and collision avoidance handled the overlap later.

The current behavior is safer and easier to explain:

- valid slot and free space: add the shuttle;
- valid slot but occupied: reject the command;
- invalid slot: reject the command.

9.9 Simple collision avoidance added

The current collision behavior is a kinematic guard. It does not simulate physical contact. Instead, it detects that another shuttle is within `0.33 m`. If movement would cross into that unsafe distance, the node reduces the step and leaves the moving shuttle waiting at the last safe pose.

This is enough for the current phase because the purpose is to prevent shuttles from visually merging and to model the expected "stop when touching another shuttle" behavior.

9.10 Room-only and full-floor compatibility

The same shuttle node works in both launch modes. The only important world parameter changes are:

- Room-only world: `-p gazebo_world_name:=room_315_only`;
- full-floor world: `-p gazebo_world_name:=mfja_3rd_floor`.

The bridge, set-pose service, and spawn service all depend on that world name. If the world name is wrong, the node can publish ROS pose topics but Gazebo will not move the visible shuttle model.

9.11 Shuttle mesh and pose alignment

The shuttle STL was updated from the corrected model file. The important practical lesson is that changing a mesh origin in CAD does not always produce the expected model-frame behavior in Gazebo if the SDF visual, collision, or link pose still applies an offset. For operational alignment, the accepted path calibration was applied to the CSV path data, and the node also keeps runtime pose calibration parameters for future tests.

The normal startup command should now run without custom scale or offset parameters.

Chapter 10

Developer Notes for Future Work

10.1 Why stoppers are not blocks yet

The current stopper implementation is a local stop primitive. It stops a shuttle at a configured point before a switch. It does not yet reserve long sections of track. Full block occupancy is a later feature. That future layer should decide whether a shuttle is allowed to enter a block based on downstream occupancy, route state, and possibly priority rules.

10.2 Why collision avoidance is not full traffic control

Collision avoidance only checks physical spacing between current shuttle poses. It is reactive. It prevents overlap but does not plan traffic. A future block system should be proactive: it should prevent a shuttle from entering a region that will become unsafe later.

10.3 Recommended next implementation phase

The recommended next phase is:

1. keep the current four start slots unless new measured entry points are accepted;
2. add named blocks to `rail_network.yaml`;
3. assign every rail segment to one or more blocks;
4. publish block occupancy in the state topic;
5. prevent shuttles from entering occupied protected blocks;
6. keep stoppers as the physical stop primitive before switches;
7. keep switch routing separate from stopper and block logic.

This preserves the clean separation already established:

```
geometry -> topology -> routing -> stopper control -> block occupancy -> Gazebo pose output
```

Chapter 11

Conclusion

The current repository is no longer just a static Gazebo map and no longer a single-package experiment. It is a ROS 2 meta-repository with dedicated description, control, bringup, and compatibility packages. The multi-robot layer provides independent namespaced control for several robot families. The Room 315 layer provides a kinematic-first shuttle system built on explicit rail topology, switch routing, stoppers, sensor-style workflow events, runtime spawning, and per-shuttle control.

The most important engineering result is separation of responsibilities: geometry is stored in package-local assets, topology is stored in `rail_network.yaml`, robot deployment is stored in YAML registries, runtime behavior is implemented in ROS 2 nodes, and the operator interface is exposed through explicit topics. This makes the project testable phase by phase, while avoiding premature contact-dynamics complexity.